

Fundamentos de Programación

Tema 8: Aplicaciones con interfaz gráfica de escritorio

Contenidos

1.- Introducción	2
2.- La programación dirigida por eventos	3
3.- Aplicaciones de escritorio en Java	4
4.- JavaFX.....	6
5.- Aplicaciones MVC	11
6.- Elementos básicos de las interfaces de usuario	17
7.- Realización de una aplicación con arquitectura hexagonal	26

Tema 8

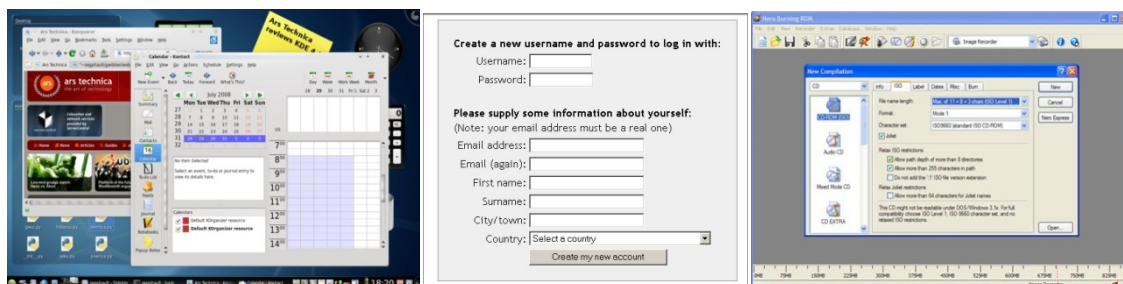
APLICACIONES CON INTERFAZ GRÁFICA DE ESCRITORIO

En este tema estudiaremos una introducción a la programación de aplicaciones con una interfaz gráfica basada en ventanas, tal y como utilizan los programas que encontramos en los ordenadores actuales.

1.- Introducción

Hoy día los ordenadores tienen instalado un sistema operativo con un entorno gráfico de escritorio, como Windows, Linux (Gnome, KDE, etc) o Mac OS. En ellos las aplicaciones se ejecutan dentro de ventanas y su interfaz consta de elementos gráficos intuitivos como botones, cuadros de texto, listas desplegables, tablas, etc.

Este tipo de programas son visualmente mucho más atractivos que los antiguos programas de consola, y además, mucho más fáciles de manejar, ya que los botones, cuadros de texto, listas, etc que aparecen son siempre los mismos en todas las aplicaciones, ya que de hecho, forman parte del propio sistema operativo¹.



Desde la aparición de Windows 95 este tipo de aplicaciones se han extendido y han sido el estándar para aplicaciones domésticas. Infinidad de programas de uso común (procesadores de texto, navegadores, antivirus, etc) presentan al usuario una ventana como interfaz. En la actualidad, la espectacularidad de los diseños que ha alcanzado la combinación HTML+CSS+JS hace que este tipo de aplicaciones se esté reinventando y tomando elementos de ellos.

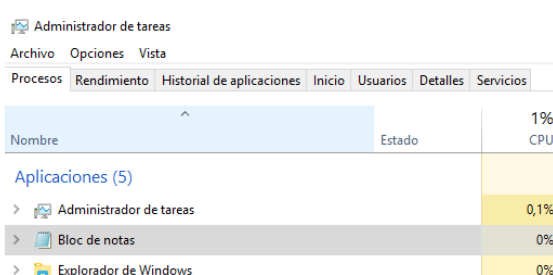
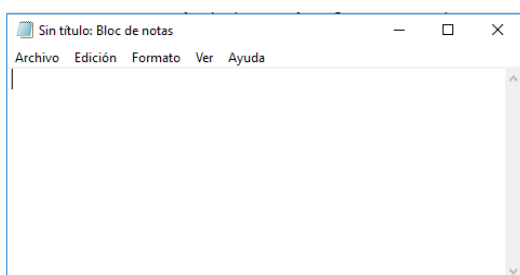
En este tema daremos una visión de cómo se han desarrollado tradicionalmente estas aplicaciones. Desde el punto de vista del programador, su desarrollo es diferente a una aplicación de consola, ya que se basa en un modelo de programación llamado **programación dirigida por eventos**.

¹ Los sistemas operativos proporcionan librerías en las que aparecen los componentes de las ventanas. Por ese motivo, las diferentes aplicaciones hacen uso de “los mismos” botones, listas, etc.

2.- La programación dirigida por eventos

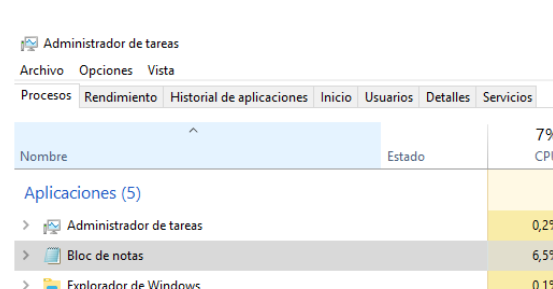
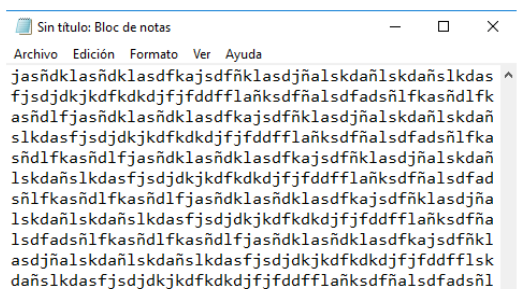
Las aplicaciones con interfaz gráfica de escritorio (de ahora en adelante, las llamaremos simplemente *aplicaciones de escritorio*) siguen un modelo de programación denominado **programación dirigida por eventos**.

Este modelo consiste en que el programa cuando se inicia, pasa al estado de espera hasta que el usuario realice alguna acción. Es decir, cuando abrimos el programa, la ventana se dibuja y automáticamente el programa “se desactiva”, sin consumir CPU. Podemos comprobarlo con un sencillo experimento. Si abrimos a la vez un programa cualquiera (por ejemplo, el bloc de notas) y el administrador de tareas, el uso de CPU del programa es 0% cuando no hacemos nada sobre él.



Nombre	Estado	1% CPU
Aplicaciones (5)		
Administrador de tareas		0,1%
Bloc de notas		0%
Explorador de Windows		0%

En cambio, cuando empezamos a escribir en el bloc de notas, podemos ver que el programa se activa y pasa a consumir CPU. Por tanto, el programa solo pasa a ejecución cuando se ha producido un **evento** sobre la ventana. El resto del tiempo está desactivado, sin consumir recursos.



Nombre	Estado	7% CPU
Aplicaciones (5)		
Administrador de tareas		0,2%
Bloc de notas		6,5%
Explorador de Windows		0,1%

Esta es una diferencia **importantísima** con las aplicaciones tradicionales, que siempre están en ejecución y se ejecutan de “arriba a abajo” hasta que terminan. Una aplicación gráfica solo se activa cuando se ha producido un evento que le afecta, y además, lo hace ejecutando un bloque de código llamado **manejador de evento (event handler)** que tiene las líneas de código que responden a ese evento. Es decir, el manejador de eventos, contiene la respuesta del programa al evento.

Hay muchos eventos posibles: pulsar el ratón en un botón, seleccionar un elemento en una lista, mover el ratón encima de una imagen, maximizar la ventana, etc. La aplicación de escritorio se programa añadiendo un manejador de evento para cada evento que nos interese gestionar. Por ejemplo, si la aplicación tiene que hacer algo cuando la ventana se maximiza, programaremos un manejador de eventos para cuando esa acción suceda.



Por último, indicamos que el programador no tiene que preocuparse de detectar los eventos, ya que eso lo hace el sistema operativo. El sistema operativo detecta los eventos y se los notifica a la aplicación, que solo deberá responder a los eventos que le interesen, programando para ellos, un manejador de eventos.

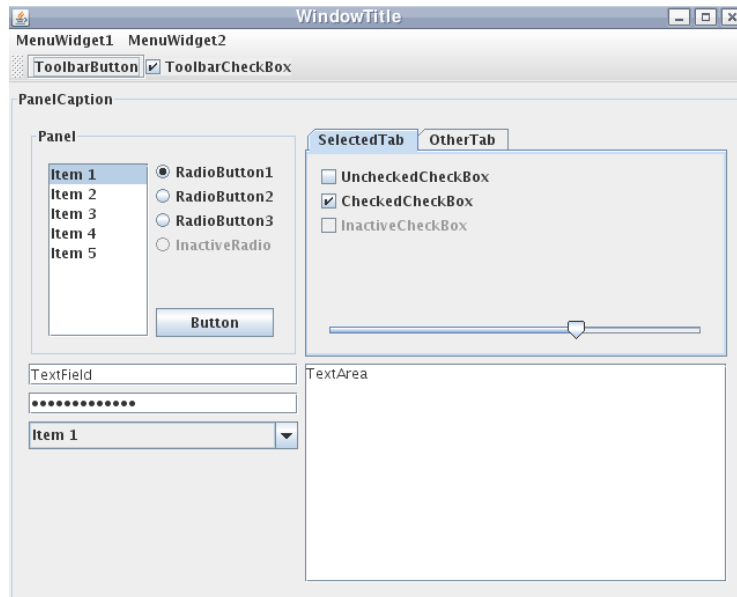
3.- Aplicaciones de escritorio en Java

En Java hay varias librerías para realizar aplicaciones de escritorio:

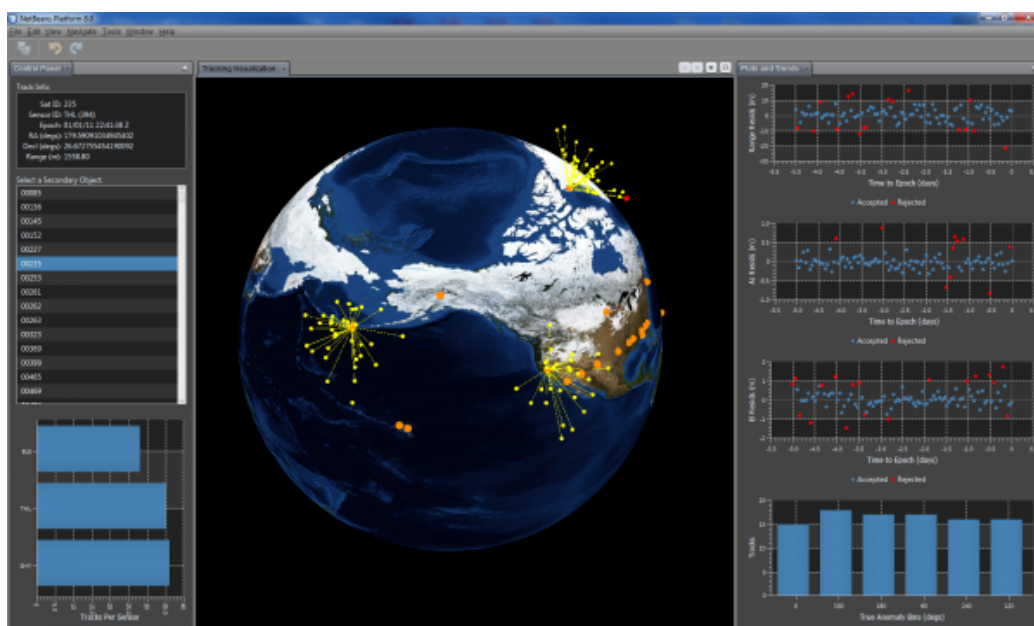
- **Awt** (1995): Con el nacimiento de Java, esta fue la primera librería que se incorporó al lenguaje para poder hacer aplicaciones de escritorio. Sus componentes eran bastante feos y además dependían del sistema operativo (en Windows se veían diferentes de Linux, por ejemplo). Hoy día no se recomienda su uso.



- **Swing** (1998): Es la librería que reemplazó a awt y sus componentes están programados íntegramente en Java, lo que garantiza que la apariencia es siempre la misma en cualquier sistema operativo. Esta librería se incluye con todos los JDK actuales y se considera la librería oficial para programar aplicaciones de escritorio. Swing todavía se usa, ya que es rápida y fácil de manejar, pero posee numerosos defectos que la hacen aconsejable solamente para interfaces simples. Por ejemplo, si queremos hacer un programa que necesite tener disponible una interfaz rápida sin complicaciones podemos usarla sin problema. En cambio, si necesitamos una interfaz más elaborada con animaciones, estilos CSS, diseño MVC, etc entonces se queda muy corta. Sus componentes, aunque útiles y funcionales, tienen un aspecto anticuado.



- JavaFx (2007):** El paso del tiempo y la gran cantidad de proyectos desarrollados con Swing han mostrado sus carencias y limitaciones. Esto, unido a los avances en el desarrollo web, donde las interfaces basadas en HTML, CSS y JS han alcanzado resultados espectaculares, y también a los avances en las técnicas de diseño de software, pusieron sobre la mesa la necesidad de que Java contase con una librería nueva para realizar interfaces de usuario profesionales como las que encontramos en la actualidad. El resultado es JavaFX, que es una librería muy completa para diseñar interfaces MVC, con CSS, animaciones, elementos 3D... Incluso es posible hacer con ella juegos y aplicaciones para móvil y Raspberry. Debido a su complejidad y a la presencia de componentes dependientes del sistema operativo, esta librería no forma parte del JDK, sino que debe ser descargada de forma independiente. La empresa **Gluon** es la encargada de su desarrollo.



4.- JavaFX

En este tema vamos a ver una introducción a las posibilidades de JavaFX para la realización de aplicaciones de escritorio. Como es lógico, no podemos tratar aquí todas ellas, ya que se requeriría un curso completo. Así que nos centraremos en el desarrollo de interfaces muy sencillas.

4.1.- Inicio de un proyecto JavaFX

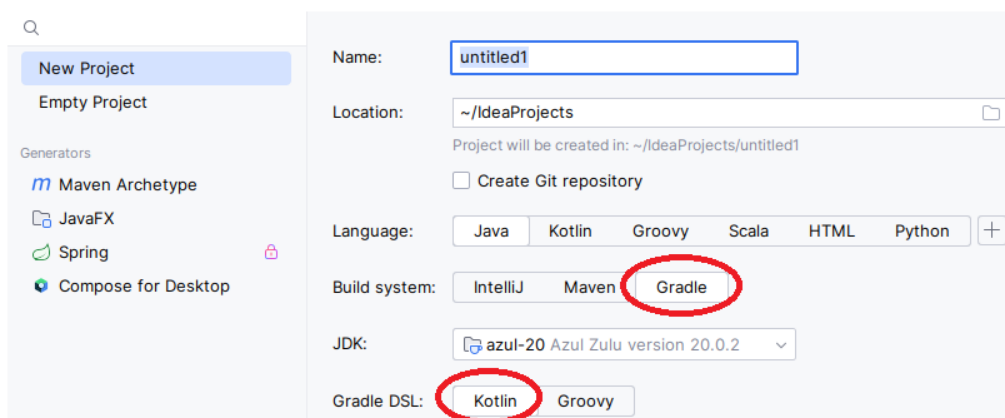
Lo primero que vamos a ver es cómo comenzar a hacer un proyecto de JavaFX. La forma más fácil, es utilizar el **sistema de construcción de proyectos Gradle**

☞ **¿Qué es Gradle?** Gradle es un sistema para construir proyectos. Es la herramienta que organiza en carpetas los archivos y librerías del proyecto y se encarga de compilar y ejecutar el programa cuando le damos al icono de “play” en el IDE.

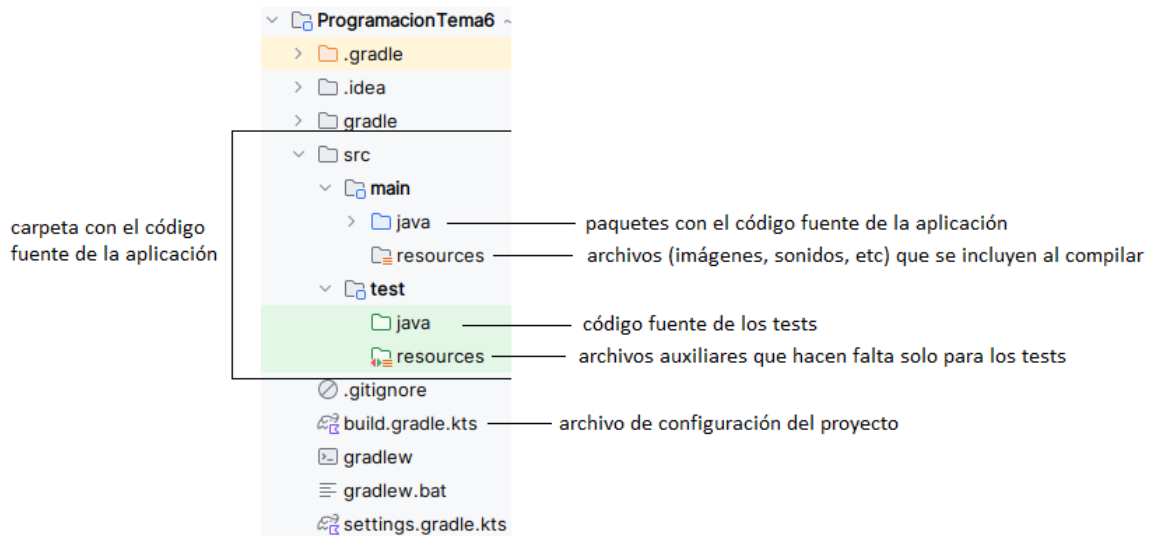
Gradle utiliza un archivo llamado **build.gradle.kts** donde se define la configuración del proyecto. Entre otras cosas, allí indicamos las librerías que vamos a utilizar, y Gradle se encarga de descargarlas y configurarlas. Cada vez que modificamos dicho archivo, debemos pulsar el icono del “elefante” 🐘 para sincronizar los cambios.

Podemos lanzar el proyecto, tests, compilar, generar javadoc, hacer un jar, etc, mediante las **tasks** que define Gradle.

Para comenzar un proyecto JavaFX abrimos IntelliJ y en la ventana para crear el proyecto elegimos como sistema de construcción **Gradle** y elegimos **Kotlin** como su lenguaje.



Una vez que pulsamos aceptar, veremos que la estructura del proyecto de Gradle cambia un poco con respecto a la estructura de los proyectos que hemos hecho hasta ahora (que usan el sistema de construcción de IntelliJ). Podemos ver que se forman carpetas separadas para el código fuente (**src**) y los tests (**test**). También podemos ver que aparece una carpeta **resources** donde podremos añadir las imágenes y otros archivos que necesite nuestra aplicación, y que se incluirán en el ejecutable al compilar.



- paquetes con el código fuente de la aplicación

código fuente de los tests

archivo de configuración del proyecto

A continuación, veremos que se abre nuestro proyecto con un archivo llamado **build.gradle.kts**, que es donde configuraremos nuestro proyecto. En este archivo es donde indicaremos que nuestro proyecto es de JavaFX. Para ello, modificamos el archivo para hacerle estos cambios:

El archivo final deberá quedarse de esta forma:

```

1.  plugins {
2.      id("application")
3.      id("org.openjfx.javafxplugin") version "0.1.0" // plugin de javafx para Gradle
4.  }
5.  group = "org.example"
6.  version = "1.0-SNAPSHOT"
7.  repositories {
8.      mavenCentral()
9.  }
10. application{
11.     mainClass="dam.tema6.ProgramaEjemplo" // aquí ponemos la clase (con su paquete) que tiene el main
12. }
13. javafx{
14.     modules("javafx.controls","javafx.fxml") // módulos que usaremos
15.     version="21" // versión de JavaFX a utilizar
16. }
17. dependencies {
18.     testImplementation(platform("org.junit:junit-bom:5.9.1")) // otras librerías
19.     testImplementation("org.junit.jupiter:junit-jupiter")
20. }
21. tasks.test {
22.     useJUnitPlatform()
23. }

```

Cuando hemos modificado el archivo de configuración y sincronizamos los cambios pulsando el icono del elefante  (si no sale, podemos darle al botón  del menú Gradle), veremos cómo se descargan e instalan todas las librerías necesarias de JavaFX, junto con sus dependencias. Esto es muy cómodo, porque el proceso de añadirlas y configurarlas a mano es complejo².

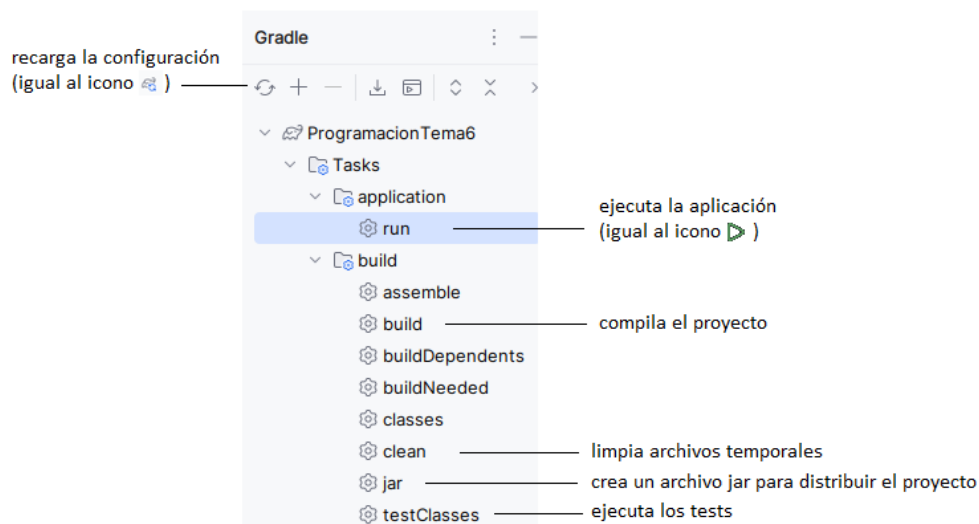
Una vez que todas las librerías se han descargado (puede tardar unos minutos), podemos crear nuestro programa JavaFX de esta forma:

- Hacemos un paquete dentro de la carpeta **main/Java**
- Pulsamos el botón derecho del ratón en ese paquete y elegimos **new** → **JavaFX Application**
- En la ventana emergente que aparece, escribimos el nombre de la clase que tendrá el main
- Es muy importante que el paquete y clase coincidan con lo que hemos puesto en el **mainClass** dentro del archivo **build.gradle.kts**

Veremos que se genera un archivo con este código fuente:

```
1. package dam.tema6;
2. import javafx.application.Application;
3. import javafx.stage.Stage;
4. public class Programa1 extends Application {
5.     public static void main(String[] args) { // este es el main de la aplicación JavaFX
6.         launch(args); // el método launch inicializa JavaFX y pasa el control al método start
7.     }
8.     @Override
9.     public void start(Stage primaryStage) {
10.         // aquí vendrá el código fuente de la aplicación JavaFX
11.     }
12. }
```

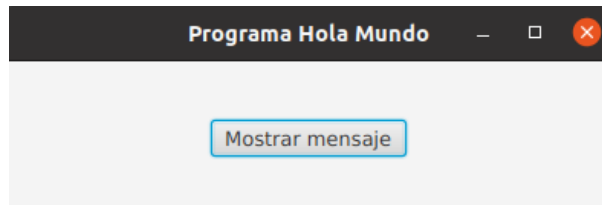
A partir de este momento, podremos hacer uso de las tareas (**tasks**) de Gradle que nos aparecen en el menú Gradle:



² En <https://openjfx.io/openjfx-docs/> se puede consultar cómo instalar JavaFX para los diferentes IDE usando varios sistemas de construcción de proyectos diferentes.

4.2.- Elementos básicos de una aplicación JavaFX

Una vez que hemos iniciado el proyecto en JavaFX, vamos a hacer una sencilla aplicación que abra una ventana con un botón, que al pulsarlo nos muestre en una ventana emergente el mensaje “hola mundo” en ella. Con este programa tan sencillo vamos a ver cómo es la estructura de las aplicaciones JavaFX.

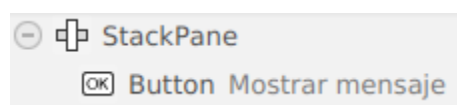


En primer lugar, JavaFX muestra en pantalla una ventana (**Stage**), en la que aparece dibujada una escena (**Scene**) con los elementos (**Node**) que forman la interfaz de usuario. Todos esos elementos deben encontrarse dentro de un elemento contenedor principal (**Parent**). En este ejemplo, nuestra ventana tiene dos elementos:

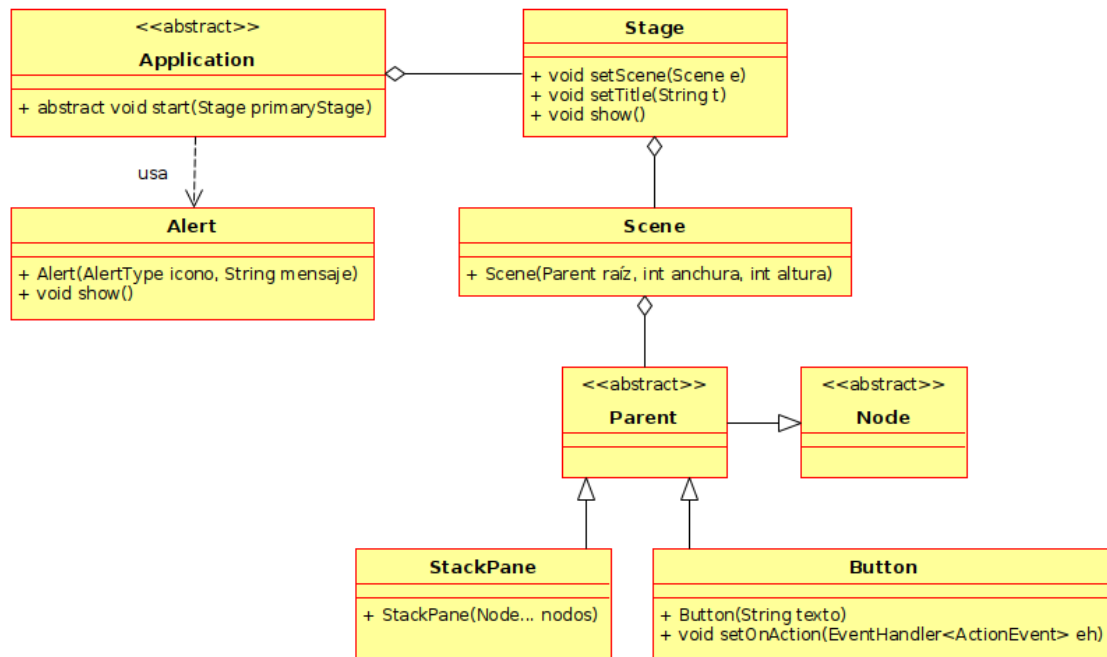
- Un panel, que es la zona de color gris que sirve de base a ventana, y sobre la que se disponen los demás elementos. Como veremos, hay muchos tipos de paneles y el que usamos aquí es un **StackPane**, en el que los elementos se disponen de forma “apilada” centrados en él.
- Un botón, que al ser pulsado lanzará la ejecución de una expresión lambda con el código fuente que se ejecutará como respuesta a la pulsación.



Por tanto, la ventana (Stage) contiene una escena (Scene), y los elementos de esta se disponen en forma de un árbol, de manera que hay un elemento raíz (en este caso, el panel) que contiene a todo lo demás (en este caso, el botón). Todas las escenas de JavaFX se dibujarán siempre de esta manera jerárquica, donde existe un elemento raíz que contiene al resto. Podemos ver gráficamente el diseño de la escena en esta imagen:



Todos los elementos descritos son clases que forman el siguiente diagrama:



Por tanto, para programar la aplicación del ejemplo, seguiremos los siguientes pasos, dentro del método **start** de nuestro programa:

- Crearemos un Button y un StackPane, de forma que añadiremos el Button al StackPane en su constructor.

```

1. @Override
2. public void start(Stage primaryStage) {
3.     Button btnMostrarMensaje = new Button("Mostrar mensaje");
4.     StackPane panel=new StackPane(btnMostrarMensaje);
5. }
  
```

- Creamos un objeto **Scene**, que tendrá como elemento principal el panel, y le pasaremos el tamaño de la escena, que será de 200x100 píxeles

```

1. @Override
2. public void start(Stage primaryStage) {
3.     Button btnMostrarMensaje = new Button("Mostrar mensaje");
4.     StackPane panel=new StackPane(btnMostrarMensaje);
5.     Scene e = new Scene(panel,400,100);
6. }
  
```

- Añadimos la escena creada a la ventana de la aplicación, que está representada en el parámetro **primaryStage** que recibe el método start. Aprovecharemos para ponerle el título a la ventana (método setTitle) y mostrarla (método show)

```

1. @Override
2. public void start(Stage primaryStage) {
3.     Button btnMostrarMensaje = new Button("Mostrar mensaje");
4.     StackPane panel=new StackPane(btnMostrarMensaje);
5.     Scene e = new Scene(panel,400,100);
6.     primaryStage.setTitle("Programa Hola Mundo");
7.     primaryStage.setScene(e);
8.     primaryStage.show();
9. }
  
```

- Por último, configuraremos el Button, de manera que al ser pulsado, se lance una ventana emergente con el mensaje “hola mundo”. Esto se hará, pasando al método `setOnAction` del Button una expresión lambda con el código fuente encargado de lanzar la ventana emergente. A tal fin, se usará la clase **Alert**, que muestra ventanas emergentes

```

1.  @Override
2.  public void start(Stage primaryStage) {
3.      Button btnMostrarMensaje = new Button("Mostrar mensaje");
4.      btnMostrarMensaje.setOnAction( evt -> {
5.          new Alert(Alert.AlertType.INFORMATION, "Hola mundo").show();
6.      });
7.      StackPane panel=new StackPane(btnBoton);
8.      Scene e = new Scene(panel,400,100);
9.      primaryStage.setTitle("Programa Hola Mundo");
10.     primaryStage.setScene(e);
11.     primaryStage.show();
12. }

```

5.- Aplicaciones MVC

Las interfaces de usuario de JavaFX pueden programarse en el código fuente del método `start`, tal y como hemos visto en el ejemplo del apartado anterior. Sin embargo, esto es muy poco práctico ya que en programación nunca es buena idea “mezclar” cosas, y cuando creamos los objetos de la interfaz en el código, estamos mezclando el diseño de la interfaz con la lógica de la aplicación, que sería la parte del código que realiza las funciones del programa.

En los programas profesionales es bueno que ambas partes estén completamente separadas, de manera que el diseño de la interfaz esté en archivos diferenciados de la parte del código de la aplicación. De esa forma, se simplifica el diseño y mantenimiento de la aplicación y además, las interfaces pueden ser realizadas por profesionales del diseño gráfico.

Una de las formas de hacer esta separación es el patrón **modelo-vista-controlador**³ (MVC), que consiste en dividir la aplicación en tres partes:

- **Modelo:** Son las clases que realizan las tareas de nuestro programa, independientemente de la interfaz de usuario o la forma de la aplicación (escritorio, web o móvil). Todos los diagramas realizados en el tema 5 serían el modelo de sus correspondientes aplicaciones.
- **Vista:** Son los diseños de las pantallas de la aplicación. Pueden estar escritos en un lenguaje diferente al del programa. Por ejemplo, en JavaFX las vistas son archivos en formato **FXML**, mientras que en Android las vistas son archivos **XML**, y en aplicaciones web las vistas son archivos **HTML**⁴
- **Controlador:** Son clases que procesan la interacción del usuario con las vistas y utilizan las clases del modelo para realizar las funciones de la aplicación en el momento adecuado.

³ Este patrón, aunque ya algo antiguo, es muy importante y se sigue usando hoy día no solo en aplicaciones de escritorio, sino también en aplicaciones web y aplicaciones para móviles. También se utilizan arquitecturas diferentes, más robustas, como el MVVM (Modelo-Vista-ViewModel) o la arquitectura hexagonal.

⁴ Los frameworks de desarrollo web pueden usar otros formatos que extienden o generan el HTML.

5.1.- Diseño de vistas con JavaFX

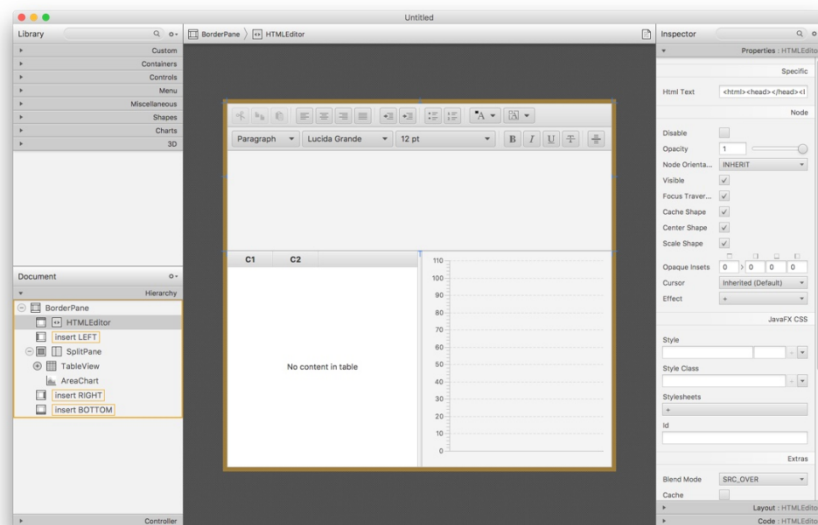
En JavaFX el árbol que forma la interfaz de usuario de una escena se puede escribir en un archivo **fxml** que luego se carga en el método **start**. De esta forma, nos evitamos tener que crear los objetos de la escena en el código y se produce la separación del MVC.

Por ejemplo, el archivo fxml correspondiente al ejemplo del apartado anterior sería este:

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <?import javafx.scene.control.Button?>
3. <?import javafx.scene.layout.StackPane?>
4. <StackPane xmlns="http://javafx.com/javafx/19" xmlns:fx="http://javafx.com/fxml/1"
5.     maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity"
6.     prefHeight="101.0" prefWidth="200.0" >
7.     <children>
8.         <Button mnemonicParsing="false" text="Mostrar mensaje" />
9.     </children>
10. </StackPane>
```

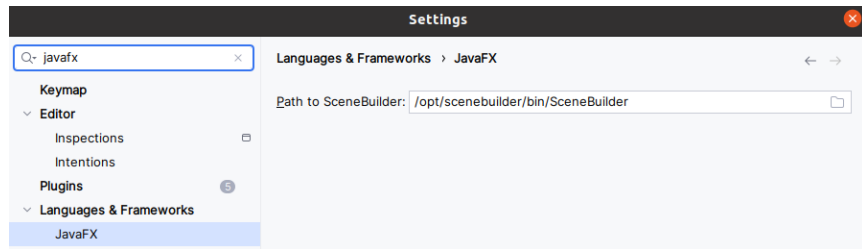
Como vemos, el archivo fxml es un archivo xml donde se definen, usando etiquetas que nos recuerdan al html, los elementos de la escena.

Sin embargo, como codificar directamente en fxml es complicado, la empresa Gluon ha desarrollado una aplicación gratuita llamada **SceneBuilder** con la que podemos diseñar de forma gráfica las vistas de la aplicación y obtener el archivo fxml correspondiente.

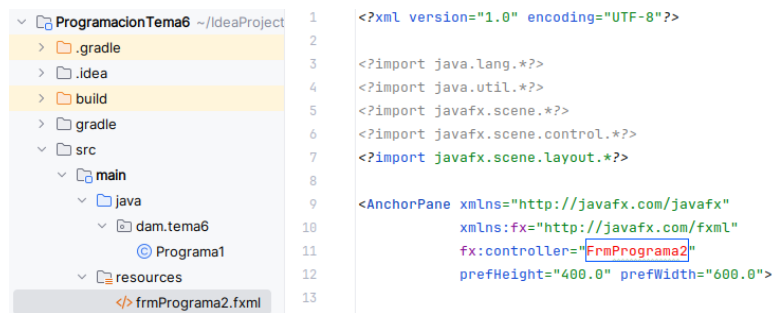


Podemos descargar dicha aplicación desde <https://gluonhq.com/products/scene-builder/> . Allí podremos encontrar la versión apropiada para nuestro sistema operativo e instalarla con el procedimiento habitual.

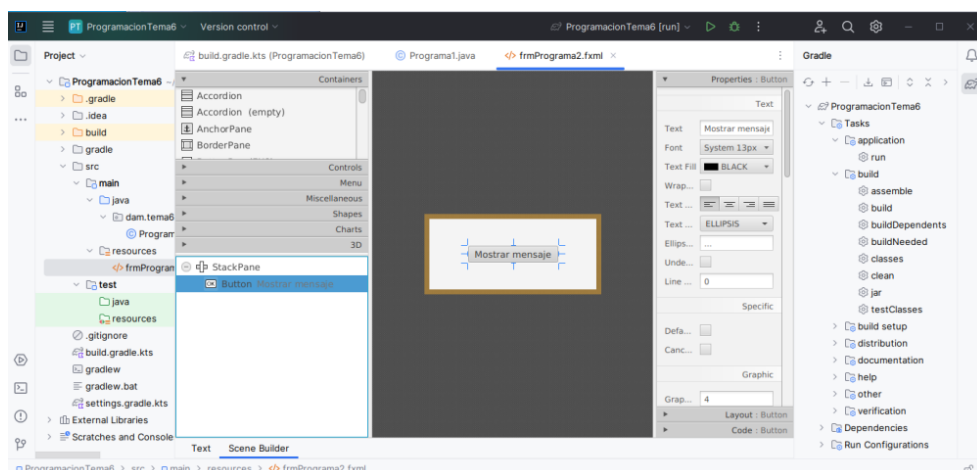
Aunque podemos usar Scene Builder como un programa independiente que sirve para diseñar vistas en formato fxml, es muy útil integrarlo con IntelliJ para así poder usarlo sin salirnos del IDE. Bastará con ir a **settings** y buscar el menú JavaFX. Allí pondremos la ruta a donde tengamos instalado SceneBuilder.



Vamos a repetir el ejemplo del apartado anterior, pero ahora usando el modelo MVC. Comenzaremos creando la vista, y para ello, pulsaremos el botón derecho en la carpeta **Resources** y elegiremos **new → FXML File**. Pondremos el nombre de la vista (es costumbre que comience por **frm**) y veremos que se abre un archivo fxml con una plantilla sobre la que podemos trabajar. Podemos ignorar el error que nos aparece.



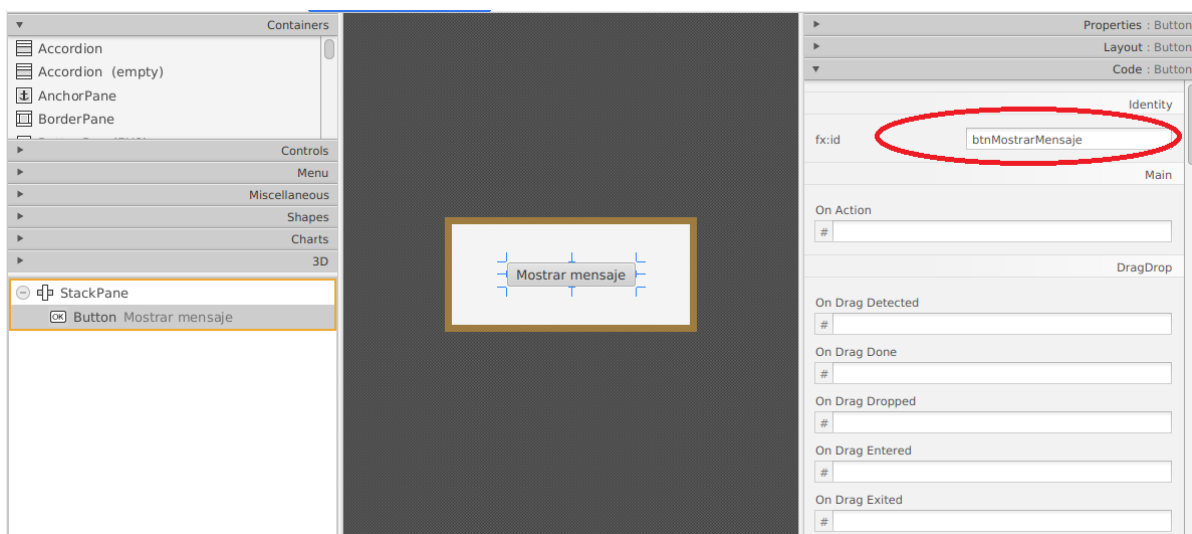
En la parte inferior de la pantalla veremos una pestaña que nos permite alternar entre la vista de código fuente fxml y el programa SceneBuilder. Pulsándola, SceneBuilder se abrirá dentro de IntelliJ⁵ y podremos editar de forma visual el contenido del archivo arrastrando componentes. No obstante, esta integración aún está incompleta y SceneBuilder pierde funcionalidad si se usa así. Lo mejor es pulsar el botón derecho del ratón en el fxml y elegir **Open in SceneBuilder** para que este se abra como programa independiente, conservando todas sus funciones.



⁵ Es posible que la primera vez nos de un error porque necesita instalar el **SceneBuilder toolkit**, pero allí mismo encontramos un botón que nos permite instalarlo y el problema se soluciona.

Para diseñar la interfaz de nuestro ejemplo, seguiremos estos pasos:

- Borramos el AnchorPane que nos generó IntelliJ por defecto
- Desde la sección Containers arrastramos un StackPane al interior de la ventana
- Cambiamos el tamaño del StackPane para que tenga aproximadamente 200x100
- Desde la sección Controls arrastramos un Button al interior del StackPane
- Seleccionamos el botón y en la sección derecha desplegamos la sección **Properties**. En el cuadro **Text** escribimos “Mostrar mensaje”
- Con el botón seleccionado desplegamos la sección **Code**. Al principio, veremos el cuadro de texto fx:id, y pondremos en él **btnMostrarMensaje**. Este es el identificador que tendrá el botón, y es muy importante para luego poder acceder a él desde el código fuente.



5.2.- Programación del controlador

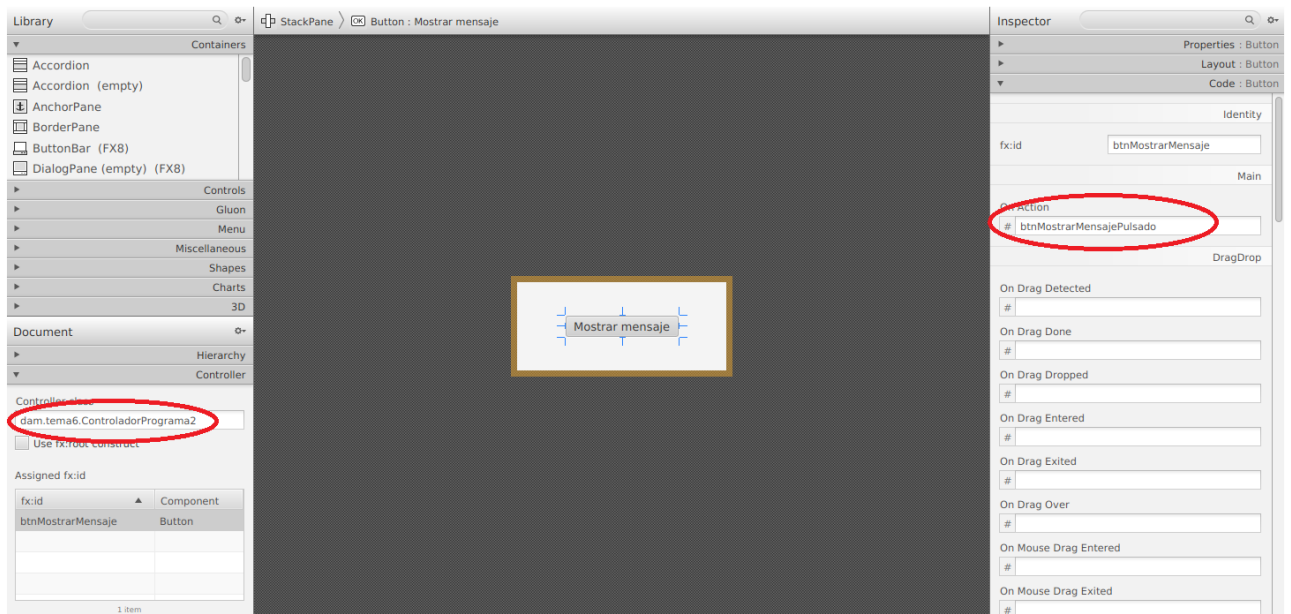
Con SceneBuilder diseñamos el archivo fxml que forma la escena (o vista) que se mostrará en la aplicación. El siguiente paso es programar un “controlador” para esa vista. El controlador es una clase Java que se encarga de dar vida a los componentes de la vista. Allí es donde estará definido el código fuente que se activa cada vez que se pulse un botón de la vista, se seleccione alguna opción en una lista, el usuario arrastre algo en algún lugar, etc. Cada vez que el usuario interactúe con un elemento de la vista, se ejecutará el código fuente de algún método⁶ definido en el controlador.

El mismo SceneBuilder nos puede ayudar a programar el “esqueleto” del controlador.

- Nos vamos a la zona inferior izquierda y desplegamos la sección **Controller**. Allí veremos el cuadro de texto Controller class, y pondremos el nombre (con su paquete) de la clase que actuará como controlador de esta vista (en estos apuntes usamos el nombre **dam.tema6.ControladorPrograma2**)

⁶ Como hemos indicado, este tipo de métodos se denominan **manejadores de eventos**

- En la interfaz que hemos dibujado en SceneBuilder seleccionamos el botón **btnMostrarMensaje**.
- Nos vamos ahora a la zona derecha y desplegamos la sección **Code**. Allí veremos un montón de cuadros de texto vacíos que se corresponden con las distintas acciones o eventos que el usuario puede realizar con el botón. Uno de ellos es **onAction**, que es el evento que se produce cuando el usuario pulsa el botón.
- Vamos a añadir al controlador un método que se active cuando se pulsa el botón. Escribimos en **onAction** el nombre de dicho método (aquí lo llamaremos **btnMostrarMensajePulsado**). De esta forma, vemos cómo el controlador tendrá los métodos que rellenemos en la sección Code para los diferentes eventos.



- A continuación, en SceneBuilder nos vamos al menú **View** y elegimos **show sample controller skeleton**, y se abrirá una pantalla con el esqueleto (stub) del controlador.

```

Sample Skeleton for 'frmPrograma2.fxml' Controller ...

package dam.tema6;

import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.scene.control.Button;

public class ControladorPrograma2 {

    @FXML
    private Button btnMostrarMensaje;

    @FXML
    void btnMostrarMensajePulsado(ActionEvent event) {

    }

}

```

Copiaremos dicho código y lo pegaremos en una nueva clase en IntelliJ

```

1. public class ControladorPrograma2 {
2.     @FXML
3.     private Button btnMostrarMensaje;
4.     @FXML
5.     void btnMostrarMensajePulsado(ActionEvent event) { }
6. }

```

Como vemos, aparece un objeto Button llamado btnMostrarMensaje, que es justamente el id del botón que arrastramos en SceneBuilder. También aparece el método btnMostrarMensajePulsado que habíamos asociado al evento onAction del botón. Más adelante comentaremos para qué sirven las anotaciones **@FXML** que vemos en el código.

De momento, terminaremos nuestro controlador programando dentro del método el código que abre la ventana emergente:

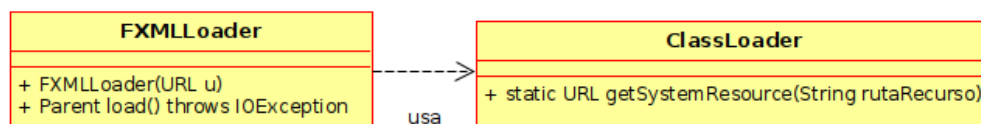
```

1. public class ControladorPrograma2 {
2.     @FXML
3.     private Button btnMostrarMensaje;
4.     @FXML
5.     void btnMostrarMensajePulsado(ActionEvent event) {
6.         new Alert(Alert.AlertType.INFORMATION, "Hola mundo").showAndWait();
7.     }
8. }

```

5.3.- Integración de la vista y el controlador en la aplicación

Como último paso para terminar la aplicación, es necesario crear en el método **start** una escena y mostrarla. El objeto contenedor (Parent) de dicha escena está definida en un archivo fxml, y las clases y métodos que se utilizan para cargarlo son:



El archivo fxml está en la carpeta **Resources** del proyecto y por tanto, la forma de cargarlo es especial (porque el contenido de esa carpeta se incluye en el ejecutable del proyecto y por tanto, no es un archivo con una ruta de archivo típica). Por eso hace falta la clase **ClassLoader**, cuyo método **getResource** permite poner la ruta del archivo (relativa a la carpeta Resources) y cargarlo.

Una vez obtenido el objeto URL bastará con pasarlo al constructor de la clase FXMMLoader dentro del método start de la clase principal de la aplicación. El nodo raíz de la escena se carga con el método load y lo podemos hacer dentro del constructor de Scene. Se hace todo así:

```

1. @Override
2. public void start(Stage primaryStage) {
3.     try {
4.         FXMMLoader fxLoader = new FXMMLoader(ClassLoader.getResource("frmPrograma.fxml"));
5.         Scene e = new Scene(fxLoader.load());
6.         primaryStage.setScene(e);
7.         primaryStage.show();
8.     } catch (IOException e) {
9.         System.out.println("Error al cargar el FXML: "+e.getMessage());
10.    }
11. }

```


☞ **¿Qué hace exactamente `FXMLLoader.load`?** Este método carga el `fxml` y se encarga de crear los objetos a partir de las etiquetas definidas en dicho archivo.

Pero no solo eso, sino que también crea un objeto con el controlador asociado a la vista. Para ello, crea el objeto y le rellena⁷ las propiedades con los objetos que ha cargado de la interfaz y para ello utiliza las anotaciones `@FXML`, para saber, por medio del atributo `fx:id`, qué etiqueta se corresponde con cada propiedad, y qué método del controlador con qué evento.

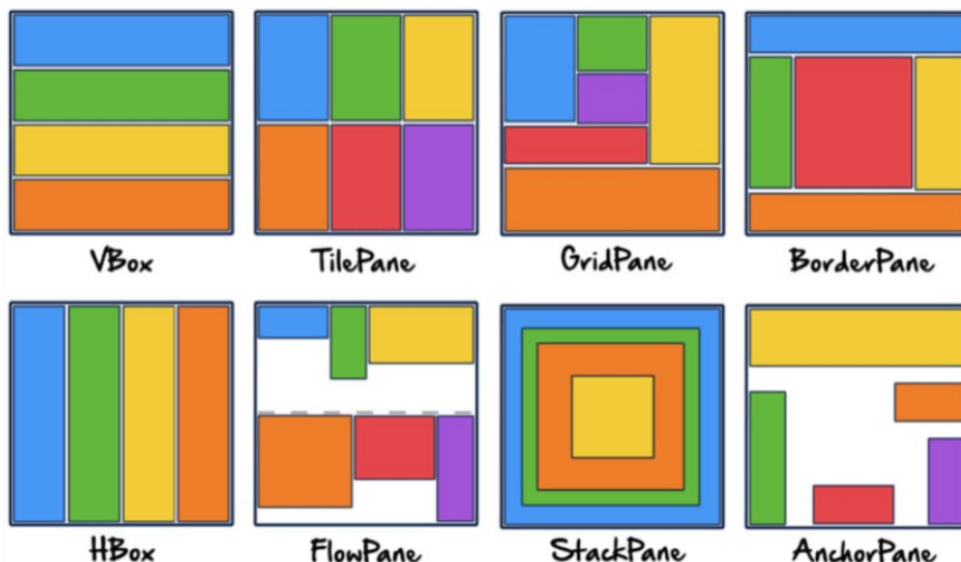
Por ejemplo, la etiqueta `<Button fx:id="btnMostrarMensaje"...>` crea un objeto `Button` que se inyecta en la propiedad `@FXML private Button btnMostrarMensaje`, y el evento definido en `onAction="#btnMostrarMensajePulsado"` se asocia al evento `onAction` de dicho botón mediante el método `@FXML void btnMostrarMensajePulsado(EventAction evt)`

6.- Elementos básicos de las interfaces de usuario

Las interfaces de usuario contienen elementos que podemos arrastrar gráficamente, y cuya interacción con el usuario podemos controlar por medio de eventos. Vamos a echar un vistazo rápido a los principales elementos que podemos incluir en la interfaz.

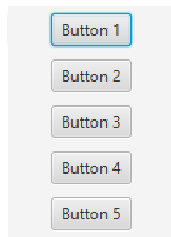
6.1.- Paneles

Los paneles son la base que forman las ventanas, de forma que los elementos de la interfaz se disponen sobre ellos. JavaFX tiene muchos tipos de paneles, que se diferencian en la forma en que se van colocando los elementos de la interfaz conforme se van añadiendo al panel.

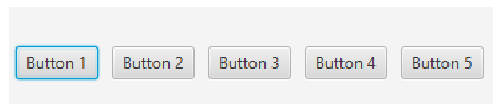


⁷ Este patrón, en el que las propiedades de una clase (en este caso, el controlador) son inicializadas de forma externa al objeto, se denomina **inyección de dependencias (DI)** y aparece sobre todo en las aplicaciones web.

- **VBox:** Es un panel en el que los elementos que se le añaden se colocan uno debajo de otro



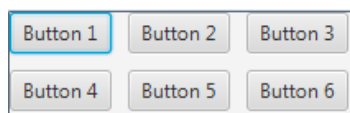
- **HBox:** Es un panel en el que todos los elementos que se le añaden se colocan uno al lado de otro, de izquierda a derecha.



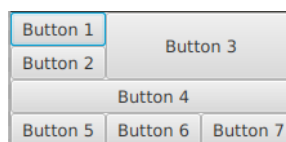
- **BorderPane:** Es un panel en el que la pantalla se divide en cinco secciones: centro, arriba, derecha, abajo e izquierda, y en cada una se coloca un elemento (normalmente, contenedor)



- **TilePane:** Es un panel que está dividido en una serie de filas y columnas de igual tamaño



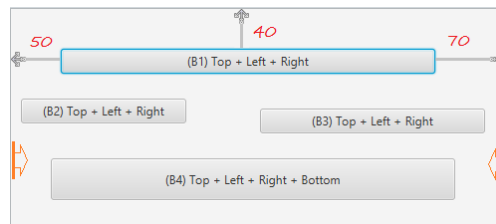
- **GridPane:** Es un panel que está dividido en filas y columnas que pueden combinarse con colspan y rowspan, como las tablas del html.



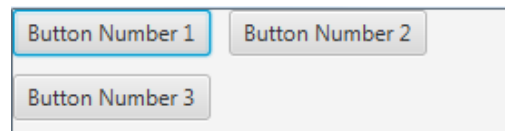
- **StackPane:** Es un panel en el que los elementos que se añaden se colocan unos encima de otros, centrados sobre el panel. Suele usarse para diseñar interfaces por **capas**



- **AnchorPane:** Es un panel en el que podemos anclar los elementos a los bordes del panel, dejando márgenes. Es adecuado cuando queremos dibujar de forma “libre” la interfaz.

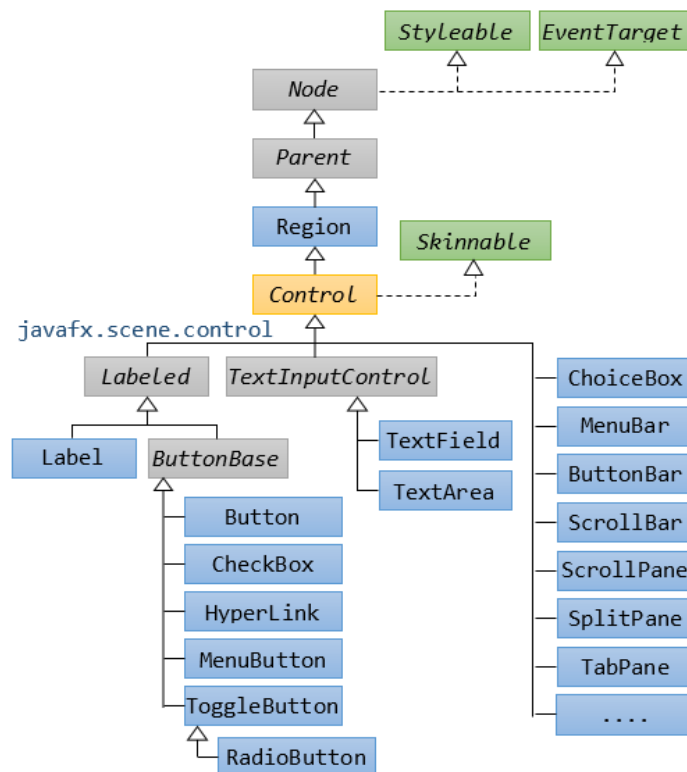


- **FlowPane:** Es un panel en el que los elementos se van disponiendo de izquierda a derecha (también se puede configurar para que sea de arriba abajo), de forma que al llegar al borde del panel, los siguientes se bajen. Es como cuando escribimos texto en un folio.



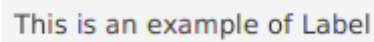
6.2.- Controles

Los controles son los elementos de la interfaz de usuario que son manejados de forma directa por el usuario. En JavaFX la clase base de todos los controles es **Control**, que recoge el comportamiento común a todos ellos. Todos los controles forman un árbol de herencia:



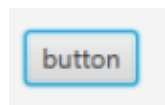
Los principales controles de las interfaces de usuario JavaFX son:

- **Label:** Son etiquetas de texto que podemos poner en cualquier lugar de la ventana. Pueden tener un papel decorativo en la aplicación, en cuyo caso no es necesario ponerles nombre a su variable. En cambio, si la etiqueta interviene de forma activa en el programa, hay que ponerle un nombre que comience por las siglas **lbl** (abreviatura de “label”).



This is an example of Label

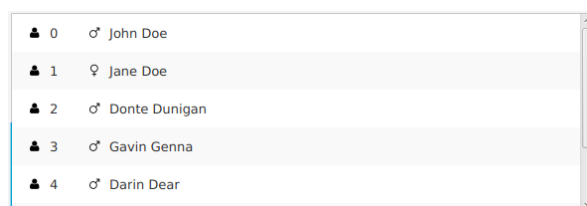
- **Button:** Son botones que el usuario puede pulsar para producir el lanzamiento de una acción. Su nombre comenzará por las siglas **btn**



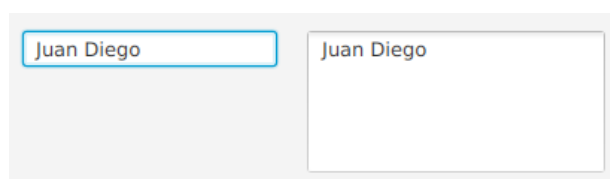
- **ComboBox<T>:** Es una lista desplegable que muestra objetos de tipo T (lo que se ve es el método toString de ellos), y en la que podemos seleccionar uno. Sus variables comienzan por **cmb**



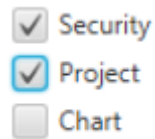
- **ListView<T>:** Es una lista que muestra objetos de tipo T y podemos ir subiendo y bajando por ella para seleccionar uno o varios. El nombre de sus variables comenzará por **lst**



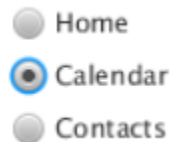
- **TextField/TextArea:** Un **TextField** es un campo de texto donde se puede escribir un renglón. Cuando queremos escribir varios renglones, en su lugar usaremos un **TextArea**. El nombre de las variables para ambos controles comienzan por **txt**



- **CheckBox:** Son casillas de verificación que podemos marcar y desmarcar. Su nombre comienza por **chk**



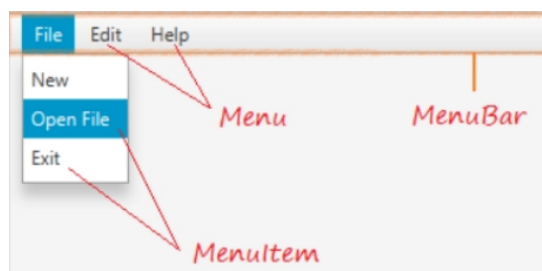
- **RadioButton:** Son como las casillas de verificación, pero excluyentes entre sí y de apariencia redondeada. Su nombre comienza por **opt**



- **ImageView:** Es un visor en el que se muestra una imagen. Su nombre comienza por **img**



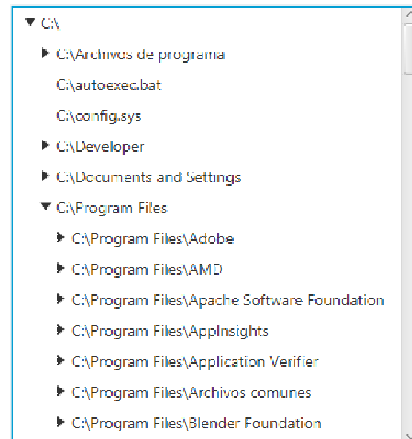
- **MenuBar:** Es una barra de menús que se coloca en la parte superior de la ventana y de él cuelgan uno o varios menús (**Menu**) con opciones (**MenuItem**).



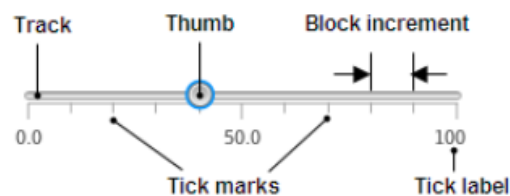
- **TableView<T>:** Es una tabla en la que cada fila muestra un objeto de tipo T. La tabla puede permitir la edición de dichos objetos. Su nombre de variable comienza por **tbl**

First Name	Last Name	Email
Jacob	Smith	jacob.smith@example.com
Isabella	Johnson	isabella.johnson@example.com
Ethan	Williams	ethan.williams@example.com
Emma	Jones	emma.jones@example.com
Michael	Brown	michael.brown@example.com

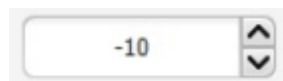
- **TreeView<T>**: Es un visor que muestra un árbol de objetos de tipo T. Su nombre de variable comienza por **tv**



- **Slider**: Es una barra deslizante que permite elegir un número. Su nombre de variable comienza por **sld**



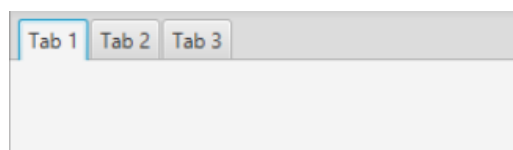
- **Spinner**: Es un cuadro de texto con botones de arriba y abajo que nos permiten elegir un número. Su nombre de variable comienza por **spn**



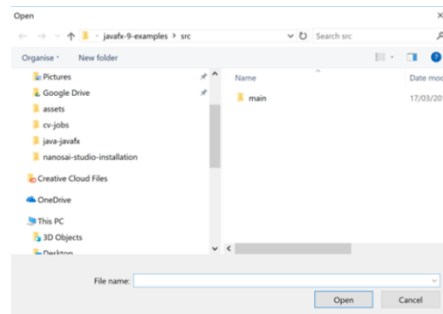
- **ProgressBar**: Es una barra de progreso que permite mostrar el avance de un proceso que se lanza en segundo plano



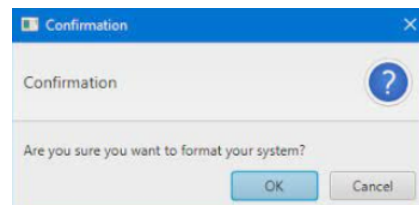
- **TabPane**: Aunque su nombre parece indicar que es un panel, en realidad es un control que tiene varias pestañas y en cada una se muestra un panel. Su nombre de variable comienza por **tab**



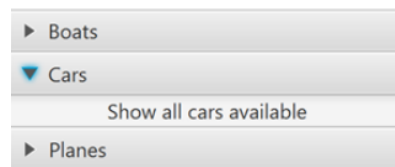
- **FileChooser:** Es una ventana para elegir archivos



- **Alert:** Es una ventana emergente que muestra un mensaje acompañado de un icono y varios botones con opciones predefinidas (ok, cancelar, etc)

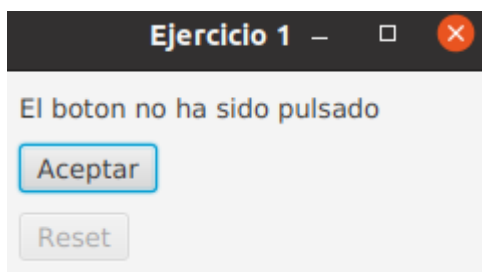


- **Accordion:** Es una zona desplegable en la que aparecen opciones. Su nombre de variable comienza por **acd**



Debido a la gran variedad de controles disponibles, remitimos a la documentación de cada uno para consultar detalladamente las posibilidades y métodos de cada uno.

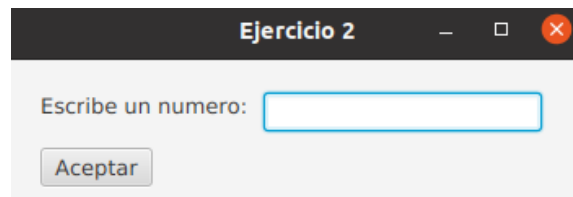
Ejercicio 1 : Haz un programa con la siguiente interfaz de usuario y comportamiento:



Métodos necesarios:

- Al iniciar el programa, el botón de reset está deshabilitado y solo está habilitado el de aceptar.
 - Al pulsar el botón de aceptar, se deshabilita el botón de aceptar y se habilita el botón de reset
 - Cuando se pulse el botón de aceptar, en la etiqueta pondrá “el botón ha sido pulsado”
 - Cuando se pulse el botón de reset, la etiqueta pondrá “el botón no ha sido pulsado”
- El **Label** tiene un método **setText** para establecer el texto de la etiqueta
 - El **Button** tiene un método **setDisable** que permite habilitarlo o deshabilitarlo

Ejercicio 2 : Haz un programa con la siguiente interfaz de usuario y comportamiento:

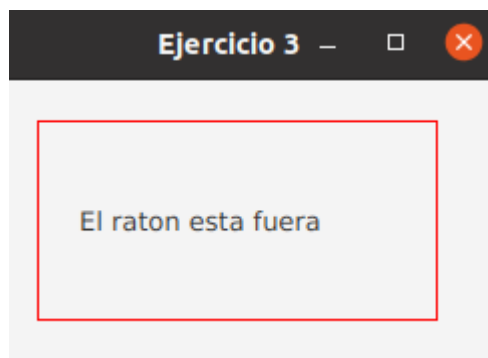


Al pulsar el botón de aceptar, se validará con una expresión regular que el usuario haya introducido un número válido, mostrando una ventana emergente de error si no es así. Si el número introducido es correcto, se mostrará una ventana emergente con el cuadrado del número.

Métodos necesarios:

- El **TextField** tiene un método **getText** para obtener el texto escrito en él

Ejercicio 3 : Haz un programa con la siguiente interfaz de usuario y comportamiento:



- La etiqueta que se ve es un **Label** que tiene tamaño 200x100 y un borde de color rojo
- Cuando el ratón entra en la etiqueta, esta cambiará y pondrá “El ratón está dentro”
- Cuando el ratón sale de la etiqueta, esta cambiará y pondrá “El ratón está fuera”

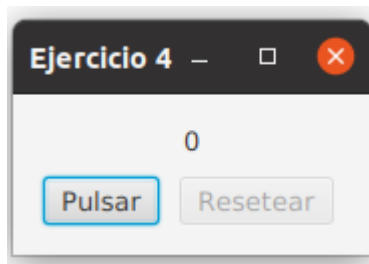
Métodos necesarios:

- El **Label** tiene un método **setPreferredSize** que permite poner el ancho y alto de la etiqueta.
- El **Label** tiene un método **setBorder**, que permite poner un borde a la etiqueta
- La clase **Border** tiene un método estático **stroke**, que crea un borde a partir de un **Paint** que se pasa como parámetro, e indica el tipo de “pintura” con la que se pinta el borde.
- La clase **Paint** tiene un método estático llamado **valueOf**, que recibe el nombre de un color (en inglés) y devuelve un objeto **Paint** que pinta con dicho color.

Eventos necesarios:

- El label tiene un evento **setOnMouseEntered**, que recibe una lambda evt ->{} que se ejecuta cuando el ratón entra en la etiqueta
- El label tiene un evento **setOnMouseExited**, que recibe una lambda evt ->{} que se ejecuta cuando el ratón sale de la etiqueta

Ejercicio 4 : Haz un programa con la siguiente interfaz de usuario y comportamiento:



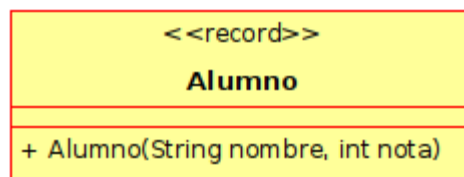
- Al principio el botón de resetear está desactivado y se activa cuando se pulsa el botón “Pulsar”
- Cada vez que se pulsa el botón “Pulsar”, se incrementará en 1 el número que está escrito en la etiqueta
- Al pulsar el botón de resetear, el contador se pondrá a 0 y se desactivará el otro botón

Cómo hacer el ejercicio:

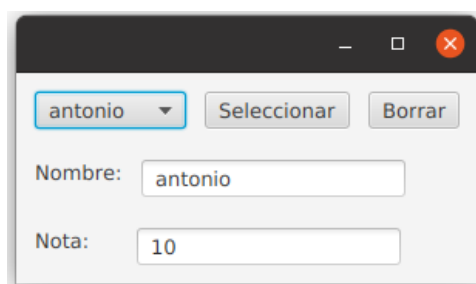
- Añadir al controlador una variable de instancia llamada **contador**, que inicialmente será 0
- Cada vez que se pulse el botón “Pulsar”, se incrementará la variable de instancia **contador**

Ejercicio 5 :

- a) Programa el siguiente record Alumno, y sobrescribe su método **toString** para que devuelva el nombre del alumno.



- b) Programa un record **Alumno** con propiedades nombre y nota, y haz un programa con la siguiente interfaz de usuario y comportamiento:



- Al iniciar la aplicación, el ComboBox aparecerá relleno con varios objetos Alumno (puedes inventártelos)
- Al pulsar el botón de seleccionar, se mostrarán en los cuadros de texto correspondientes el nombre y la nota del alumno seleccionado en el ComboBox
- Al pulsar el botón de borrar, se borrarán los cuadros de texto y no habrá ningún elemento seleccionado en el ComboBox

Métodos necesarios:

- El **ComboBox** tiene el método **getItems**, que devuelve una lista con los objetos que hay en él
- El **ComboBox** tiene el método **getSelectionModel**, que devuelve un objeto de tipo **SelectionModel**, cuyo método **getSelectedItem** nos da el objeto seleccionado en el ComboBox

7.- Realización de una aplicación con arquitectura hexagonal

En este apartado vamos a realizar de forma detallada una aplicación JavaFX de tipo **formulario maestro-detalle**, es decir, la ventana estará dividida en dos partes, de forma que en la parte izquierda se mostrará una lista de objetos, y al pulsar en cada uno se actualizará la parte de la derecha con la información detallada del objeto seleccionado. Esta es la apariencia de la aplicación que desarrollaremos:

The screenshot shows a JavaFX window titled "Carreras universitarias". On the left, there is a large empty rectangular box with a blue border, intended for a list of university degrees. To the right of this box are three text input fields labeled "Nombre:", "Duración:", and "Créditos:". Below these fields is a table with the header "Asignaturas:". The table has three columns: "Nombre", "Curso", and "Créditos". The table body is currently empty, displaying the text "Tabla sin contenido". At the bottom right of the window are two buttons: "Borrar" and "Guardar".

La aplicación muestra en la izquierda una lista con los grados universitarios que hay en una base de datos. Al seleccionar un grado podremos ver a su derecha su lista de asignaturas. Podremos crear nuevos grados, borrarlos, y también gestionar las asignaturas. También podremos pulsar en la tabla con el botón derecho y nos aparecerá un menú emergente con opciones para añadir y borrar las asignaturas de un grado. Esto nos llevará a una segunda ventana donde podremos rellenar la información de una asignatura. Dicha ventana se mostrará en **forma modal**, esto es, quedará encima de la ventana principal y no será posible volver a ella hasta que no se haya cerrado la ventana modal.

The screenshot shows a modal window titled "frmAsignaturas.fxml". It contains three text input fields labeled "Nombre:", "Curso:", and "Creditos". At the bottom right of the window is a button labeled "Guardar".

En este tema los datos estarán guardados en una **base de datos**, por lo que todo lo que hagamos con la interfaz será plenamente funcional. Cuando nos enfrentamos a este tipo de programas, deberemos plantearnos el uso de una **arquitectura** para nuestra aplicación.

7.1.- Arquitecturas de software

En general, es una buena costumbre pararse a pensar un poco antes de hacer cosas. Por ejemplo, cuando se va a construir una casa, antes de que entren los albañiles, debe haber un arquitecto que se encargue de diseñar los planos de la casa. Esa tarea de planificación hace que la construcción de la casa transcurra (teóricamente) sin imprevistos y sabiendo lo que se va a hacer. Es imposible pensar que una casa se puede construir sin dicha labor previa.

En el mundo del desarrollo de software ocurre lo mismo, y antes de ponernos a programar debemos saber qué es lo que queremos hacer y cómo lo vamos a hacer. Desgraciadamente, el desarrollo de software posee una serie de peculiaridades que hace que la labor de planificación no sea tan sencilla ni evidente. Si queremos que nuestro programa sea escalable y fácilmente mantenible, deberemos planificar en consecuencia, y esto, no siempre es obvio.

Una de las cosas que tenemos que plantearnos, es si queremos que nuestro programa tenga una **arquitectura**, o no. Una arquitectura es un diseño en el que detectamos los componentes que tendrá nuestro programa (traducidos en interfaces y clases), vemos cómo se organizarán, cómo interactuarán entre si, y si son suficientes para cumplir las necesidades presentes y futuras. Como es lógico, no usar arquitectura solo puede ser apto en programas extremadamente pequeños, que no tienen ningún tipo de ambición. Cualquier programa (incluso los pequeños), deben tener una arquitectura para que puedan tener fácil mantenimiento y posibilidades de ampliación.

Si queremos hacer el proyecto propuesto en el punto anterior sin seguir ninguna arquitectura, bastará con programar todo el código en los controladores. Cada vez que queramos conectarnos con la base de datos, grabar datos en ella, consultar cosas, etc, deberemos programar en los controladores las correspondientes sentencias, y eso hace que el código del programa se fusione irremediablemente (o lo que es lo mismo, presente alto acoplamiento) con el que gestiona los datos, de forma que sea imposible por ejemplo, cambiar el tipo de base de datos en el futuro.

Usando una arquitectura, podemos hacer que nuestro programa pueda adaptarse a cambios en las necesidades futuras, además de ser más fácil la corrección de errores y sus ampliaciones.

Un modelo de arquitectura muy actual es la **arquitectura hexagonal**, que tiene como objetivo construir programas muy modulares, que sean independientes de la interfaz de usuario, framework usado, agentes externos... de manera que cualquier aspecto que se pueda imaginar, pueda ser añadido al sistema de forma sencilla. Los programas creados de esta forma son muy flexibles, toleran mejor los cambios y son más fáciles de testear.

Podremos comprobar su potencial en este tema y en el siguiente. Aquí crearemos nuestro proyecto usando una arquitectura hexagonal mediante el enfoque clásico de una aplicación de escritorio que accede directamente a una base de datos. Veremos que este enfoque ya está en desuso y gracias a la arquitectura hexagonal, en el próximo tema lo migraremos fácilmente a un enfoque más actual basado en APIs.

7.2.- La arquitectura hexagonal⁸

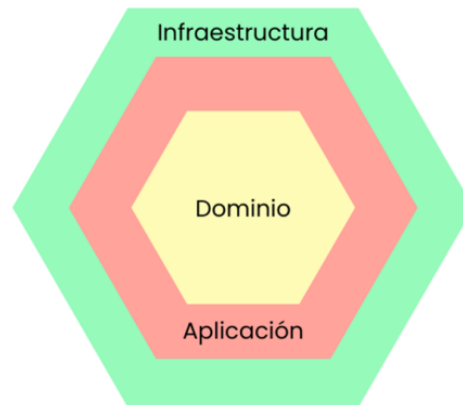
La arquitectura hexagonal es una estructuración de las clases e interfaces de la aplicación que tiene como objetivo conseguir que la funcionalidad del programa (también llamada **lógica de negocio**) esté completamente separada de las tecnologías usadas, como por ejemplo, el tipo de interfaz de usuario, los frameworks usados y los medios de almacenamiento (bases de datos, archivos, etc). Para ello, el sistema está descrito en términos de interfaces (llamadas **puertos**) que define lo que el sistema necesita conseguir, y cómo se relaciona con el exterior. Estas interfaces se implementan por clases (llamadas **adaptadores**) que utilizan tecnologías concretas.

La aplicación tendrá tres paquetes, en los que se distribuyen sus clases e interfaces:

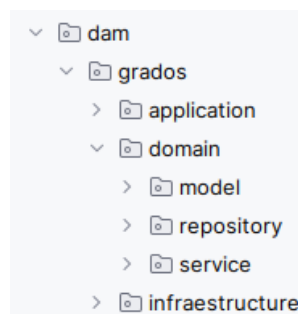
- **domain:** Está formado por estos subpaquetes (en proyectos más grandes puede haber más):
 - **model:** Aquí tendremos las clases que forman el modelo de la aplicación, o lo que es lo mismo, las clases que representan los conceptos con los que trabaja nuestro programa, de forma independiente a la interfaz de usuario y otras tecnologías que puedan considerarse.
 - **service:** Son interfaces cuyos métodos son los **casos de uso** de la aplicación. Un caso de uso es una tarea que puede realizar un usuario con el sistema. Una vez que entendemos lo que tiene que hacer nuestro programa, identificamos los casos de uso y los traducimos a métodos, que pondremos en las interfaces de este paquete.
 - **repository:** Contiene las interfaces que definen cómo son los **repositorios** de objetos. Un repositorio es un lugar donde se pueden leer y guardar objetos del modelo, y por tanto, en este paquete pondremos interfaces cuyos métodos son las operaciones necesarias para guardar y recuperar los objetos del modelo
- **application:** Contienen las clases que implementan las interfaces del paquete **service**. Por tanto, estas clases están diciendo cómo funciona el programa a nivel interno, de forma independiente de las tecnologías concretas utilizadas (interfaz de usuario, bases de datos...) En general, estos métodos indican cómo serán usados los objetos del modelo y los repositorios durante el funcionamiento del programa, estableciendo por ejemplo, las validaciones correspondientes.
- **infraestructure:** Contiene subpaquetes en los que encontraremos las implementaciones de las clases del paquete repository, clases que implementen cada tipo de interfaz de usuario que queramos considerar, la propia clase de la aplicación, etc. En general, cualquier tecnología utilizada tendrá su propio subpaquete con las clases que la implementa.

⁸ Propuesta por **Alistair Cockburn** en 2005 para evitar los problemas encontrados en el diseño orientado a objetos. Fue renombrada a **arquitectura de puertos** (las interfaces) y **adaptadores** (sus implementaciones)

Pese a su nombre, la arquitectura no tiene nada que ver con un hexágono ni con el número 6, y realmente define una estructura en capas, donde las interfaces de las capas más internas definen las cosas que “necesita” la aplicación y las capas más externas se las proporcionan. El hexágono se usa como símbolo para delimitar las capas, siendo sus lados indicadores de la variedad de situaciones con las que puede interactuar el sistema (interfaz, almacenamiento, tests, etc)



Comenzaremos creando nuestro proyecto y haremos un paquete llamado **dam.grados**, que dividiremos en los paquetes mencionados. Posteriormente iremos creando nuevos paquetes dentro de ellos.



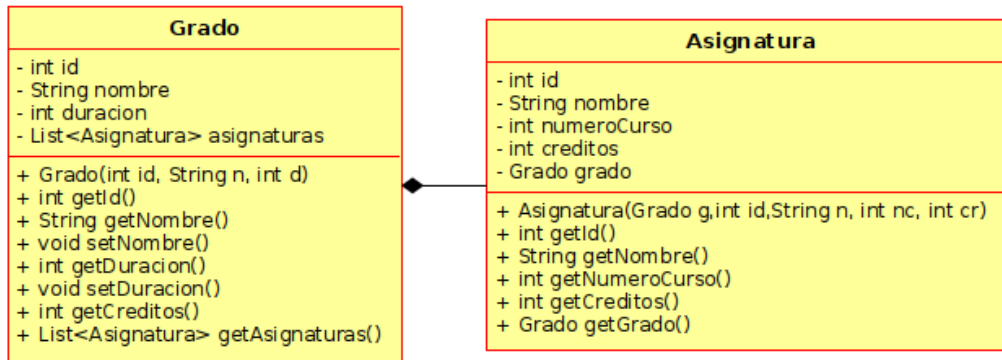
7.3.- Definición del modelo

El modelo de esta aplicación está formado por los objetos que internamente usa el programa para realizar su funcionalidad.

Como nuestro programa utiliza grados y asignaturas, definiremos dos clases para ellos. Dichas clases representarán un grado universitario y una asignatura, independientemente del lugar donde están almacenados (ram, archivos, bases de datos...), y contendrán como variables de instancia los datos que necesitaremos en nuestro programa (nombres, duraciones, etc).

Además, la clase Grado reflejará la relación de composición que se da en el mundo real entre un grado y una asignatura, donde un grado está compuesto por varias asignaturas. Este hecho se reflejará añadiendo a la clase Grado una lista con todos sus objetos Asignatura.

Añadiremos al paquete **dam.grados.domain.model** las siguientes clases:



☞ *¿Por qué no usamos records? Pueden usarse perfectamente, pero por simplicidad hemos elegido clases. Nuestro programa podrá actualizar grados, y por ese motivo la presencia de setters hace que sea más fácil trabajar con clases. Asignatura podría ser un record, pero para que sus métodos mantengan un estilo de nombres similar al de la clase Grado, seguimos usando una clase*

Comenzaremos programando la clase **Grado** de la forma habitual, teniendo en cuenta que los créditos de un grado se calculan sumando los créditos de sus asignaturas. También vamos a sobrescribir **toString** para que devuelva el nombre del grado, y los métodos **equals** y **hashCode**, de forma que dos grados se consideren iguales si tienen el mismo **id**.

```
1. public class Grado {
2.     private int id;
3.     private String nombre;
4.     private int duracion;
5.     private List<Asignatura> asignaturas;
6.     public Grado(int id, String n, int d){
7.         this.id=id;
8.         this.nombre=n;
9.         this.duracion=d;
10.        this.asignaturas=new ArrayList<>();
11.    }
12.    public int getId() { return this.id; }
13.    public String getNombre() { return this.nombre; }
14.    public void setNombre(String nombre) { this.nombre = nombre; }
15.    public int getDuracion() { return this.duracion; }
16.    public void setDuracion(int duracion) { this.duracion = duracion; }
17.    public int getCredits() {
18.        int credits=0;
19.        for(Asignatura a:this.asignaturas){
20.            credits += a.getCredits();
21.        }
22.        return credits;
23.    }
24.    public List<Asignatura> getAsignaturas() { return this.asignaturas; }
25.    @Override
26.    public String toString(){ return this.nombre; }
27.    @Override
28.    public boolean equals(Object o) {
29.        if (!(o instanceof Grado grado)) return false;
30.        return id == grado.id;
31.    }
32.    @Override
33.    public int hashCode() {
34.        return Objects.hashCode(id);
35.    }
36. }
```

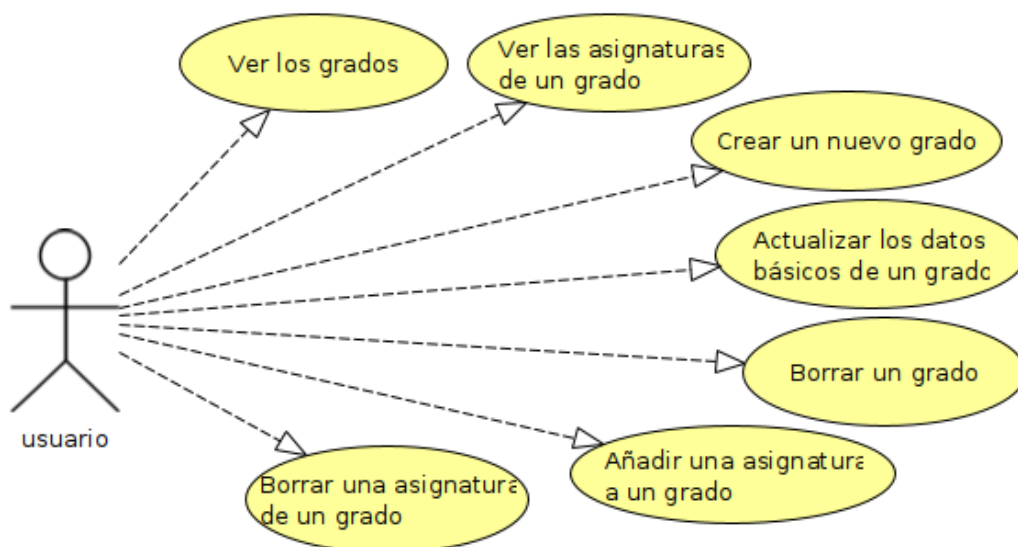
A continuación programamos la clase **Asignatura** de forma muy similar:

```
1. public class Asignatura {
2.     private int id;
3.     private String nombre;
4.     private int numeroCurso;
5.     private int creditos;
6.     private Grado grado;
7.     public Asignatura(Grado g,int id, String n, int nc, int c) {
8.         this.grado = g;
9.         this.id = id;
10.        this.nombre = n;
11.        this.numeroCurso = nc;
12.        this.creditos = c;
13.    }
14.    public int getId() { return this.id; }
15.    public String getNombre() { return this.nombre; }
16.    public int getNumeroCurso() { return this.numeroCurso; }
17.    public int getCreditos() { return this.creditos; }
18.    public Grado getGrado() { return this.grado; }
19.    @Override
20.    public String toString() {
21.        return this.nombre;
22.    }
23.    @Override
24.    public boolean equals(Object o) {
25.        if (!(o instanceof Asignatura that)) return false;
26.        return id == that.id;
27.    }
28.    @Override
29.    public int hashCode() {
30.        return Objects.hashCode(id);
31.    }
32. }
```

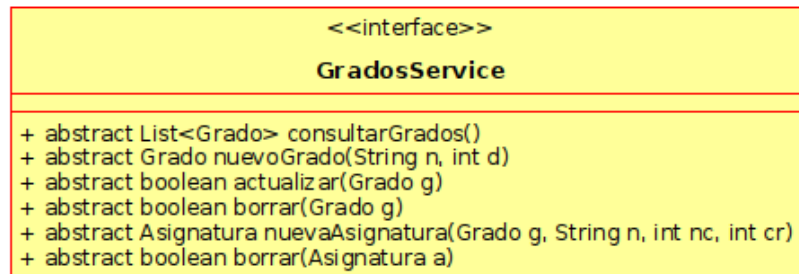
7.4.- Análisis de los casos de uso

Los **casos de uso** son las acciones que un usuario puede realizar con la aplicación. Su análisis nos ayudará a descubrir los métodos que definirán la **lógica de negocio** de la aplicación. Dichos métodos se pondrán en una (o varias, si el proyecto es muy grande) interfaces **Service**.

Puesto que el propósito de nuestra aplicación está muy claro (de hecho, tenemos una interfaz de usuario que nos dice exactamente lo que hay que hacer), es fácil obtener los casos de uso



La traducción de los casos de uso a métodos que trabajan con las clases del modelo nos permite formar la interfaz **GradosService**



Algunas observaciones sobre el diseño realizado:

- El caso de uso “ver las asignaturas de un grado” ya está recogido en el método **getAsignaturas** de la clase Grado, y por ese motivo, no aparece en **GradosService**
- Como nuestro proyecto es pequeño, es suficiente tener todas las funcionalidades en **GradosService**. Sin embargo, si la gestión de las asignaturas fuese más compleja, sería conveniente separar los métodos que trabajan con asignaturas en **AsignaturasService**. Suele ser habitual tener una clase Service por cada entidad que haya que manejar.

Por último, indicamos que en este proyecto hemos comenzado analizando el modelo y después los casos de uso, pero también hay veces en las que se hace al revés, y primero se determinan los casos de uso para después obtener el modelo. Eso se hace, por ejemplo, cuando los requisitos del sistema no están muy claros (o se está en las fases iniciales del proyecto) y es necesario comprender bien cómo interactúan los usuarios con él. También se hace en las metodologías ágiles, cuando se va a iniciar una nueva iteración del desarrollo.

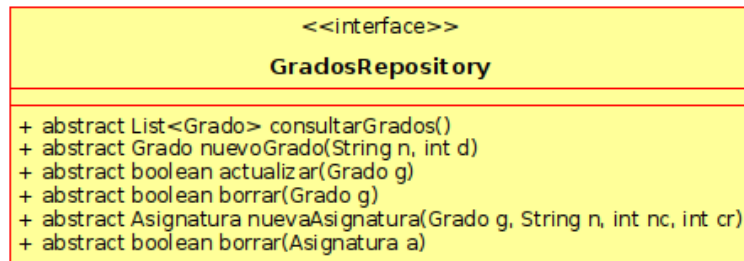
Programaremos la interfaz **GradosService** en el paquete **dam.grados.domain.service**

```
1. public interface GradosService {  
2.     public abstract List<Grado> consultarGrados();  
3.     public abstract Grado nuevoGrado(String n, int d);  
4.     public abstract boolean actualizar(Grado g);  
5.     public abstract boolean borrar(Grado g);  
6.     public abstract Asignatura nuevaAsignatura(Grado g, String n, int nc, int cr);  
7.     public abstract boolean borrar(Asignatura a);  
8. }
```

7.5.- Definición de los repositorios

Los repositorios son otra parte clave del sistema, ya que son interfaces que definen cómo se almacenan y recuperan los objetos del modelo. Por tanto, tendrán métodos que permitirán insertar, modificar, borrar y leer los objetos del modelo, según lo que necesiten los métodos identificados en la clase Service.

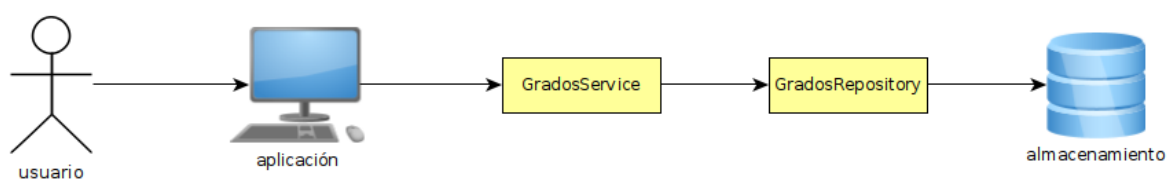
En el paquete **dam.grados.domain.repository** crearemos la interfaz **GradosRepository**, que se encargará de realizar todas las tareas relacionadas con la grabación de objetos **Grado** y **Asignatura** en el sistema de almacenamiento que usemos (en nuestro caso, será una base de datos relacional, pero este enfoque permitiría sustituirlo por una base de datos nosql si fuese necesario).



Si nos fijamos bien, los métodos de **GradosRepository** son los mismos que **GradosService**, y eso conduce a que mucha gente los confunda, no tenga clara la responsabilidad de cada uno o incluso piense que son lo mismo. Aunque los métodos de ambas clases en nuestro proyecto tienen los mismos nombres, las tareas que realizan son diferentes:

- **GradosRepository**: Los métodos se limitan a acceder al medio de almacenamiento (como la base de datos) para manipular los objetos. Por ejemplo, “guardar una asignatura en la base de datos” es algo propio de GradosRepository.
- **GradosService**: Los métodos realizan comprobaciones y validaciones impuestas por nuestra aplicación, y en consecuencia, usarán los métodos de GradosRepository cuando necesiten manipular los objetos en el sistema de almacenamiento. Por ejemplo, “se grabará una asignatura en el medio de almacenamiento si el grado asociado no tiene una asignatura con ese nombre”, es algo propio de GradosService. Normalmente, un Service trabajará con un Repository y tendrá métodos que se llamen igual (esto es algo normal, puesto que es habitual que un programa quiera gestionar los objetos que hay en un medio de almacenamiento, cosa que se hace con el Repository). Sin embargo, la lógica de negocio puede incluir más acciones que no sean solo trabajar con objetos en el medio de almacenamiento, y eso serían métodos incluidos en el Service. También podría darse el caso de métodos del Service que se puedan corresponder con varios del Repository o incluso un método del Service que pueda usar varios Repository para realizar acciones más complejas.

En general, la clase de nuestro programa será quien use los métodos de **GradosService**. A su vez, **GradosService** trabaja con **GradosRepository**, que será quien finalmente trabaje sobre el sistema de almacenamiento.

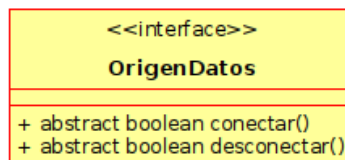


Programaremos la interfaz **GradosRepository** de la forma habitual:

```
1. public interface GradosRepository {
2.     public abstract List<Grado> consultarGrados();
3.     public abstract Grado nuevoGrado(String n, int d);
4.     public abstract boolean actualizar(Grado g);
5.     public abstract boolean borrar(Grado g);
6.     public abstract Asignatura nuevaAsignatura(Grado g, String n, int nc, int cr);
7.     public abstract boolean borrar(Asignatura a);
8. }
```

7.6.- Definición del origen de datos

El origen de datos es el sistema de almacenamiento al que nos conectamos para almacenar y recuperar la información. En nuestro proyecto consideraremos como origen de datos cualquier cosa que tenga métodos para conectarnos y desconectarnos con él.



No se pueden incluir más métodos porque la forma de guardar y recuperar la información es algo que depende demasiado de la tecnología concreta (base de datos relacional, archivos, etc).

Programaremos esa interfaz en el paquete **dam.grados.domain.repository**

```
1. public interface OrigenDatos {
2.     public abstract boolean conectar();
3.     public abstract boolean desconectar();
4. }
```

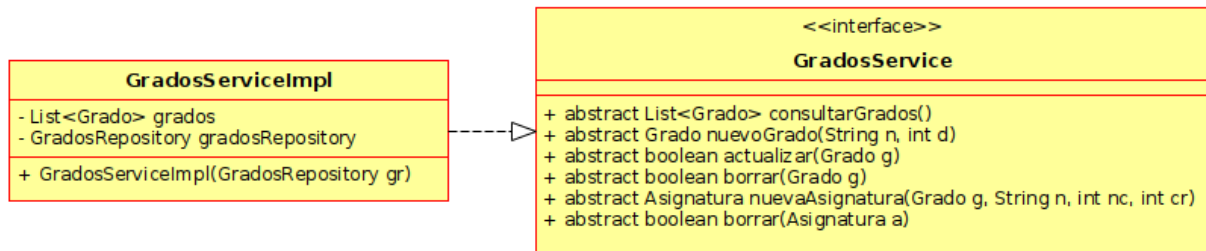
7.7.- Programación de la lógica de negocio

La **lógica de negocio** es el código que realiza las funciones requeridas por la aplicación, usando los objetos del modelo y los repositorios. La arquitectura hexagonal nos permite programar dicho código con interfaces, consiguiendo así que sea independiente de las tecnologías concretas que se utilicen (como JavaFx o MariaDB).

Como ya vimos, la interfaz **GradosService** contiene los métodos que definen la lógica de negocio. Por tanto, programar la lógica de negocio equivale a proporcionar una clase que implemente la interfaz GradosService. Llamaremos a esta clase **GradosServiceImpl** y la crearemos en el paquete **dam.grados.application**

Programar la clase **GradosServiceImpl** no es evidente, y surgen muchas posibilidades que involucran tomar decisiones que afectarán a la eficiencia, al consumo de memoria, etc. Por motivos didácticos, y para que se vea clara la diferencia entre la clase Service y Repository, optaremos por una implementación en la que los grados estarán permanentemente en la memoria (en un **ArrayList**) y solo se consultarán de la base de datos la primera vez que se quiere acceder a ellos.

Por tanto, el diseño de la clase **GradosServiceImpl** y su programación inicial es el siguiente:



```
1. public class GradosServiceImpl implements GradosService {
2.     private GradosRepository gradosRepository;
3.     private List<Grado> grados;
4.     public GradosServiceImpl(GradosRepository gr){
5.         this.gradosRepository=gr;
6.         this.grados=null; // null significa que no se han cargado los datos del almacenamiento
7.     }
8.     // aquí vienen los métodos
9. }
```

Veamos todos los métodos que programaremos en esa clase:

- **consultarGrados:** Si la lista de grados no está creada (es **null**), consultaremos todos los grados del almacenamiento y rellenaremos con ellos la lista de grados. En caso contrario, devolveremos la lista de grados.

```
1. public List<Grado> consultarGrados() {
2.     if(this.grados==null){
3.         this.grados=this.gradosRepository.consultarGrados();
4.     }
5.     return this.grados;
6. }
```

Aquí podemos ver claramente la diferencia entre Service y Repository. La clase Repository se encarga solo de consultar los datos, pero el contexto en el que deben ser cargados lo define el Service, de acuerdo al funcionamiento establecido para la aplicación.

- **nuevoGrado:** Si no existe un grado con el nombre recibido como parámetro, lo creamos en el almacenamiento, lo añadimos a la lista de grados y lo devolvemos. En caso contrario, se lanzará una **IllegalArgumentException**.

```
1. public Grado nuevoGrado(String n, int d) {
2.     for(Grado g:this.consultarGrados()){
3.         if(g.getNombre().equals(n)){
4.             throw new IllegalArgumentException("Ya existe el grado "+n);
5.         }
6.     }
7.     Grado g = this.gradosRepository.nuevoGrado(n,d);
8.     this.grados.add(g);
9.     return g;
10. }
```

Aquí vemos como el Service realiza una validación previa sobre el grado antes de encargarle al Repository que lo grabe en la base de datos. El Service se está encargando de dar cumplimiento la “regla de negocio” de que no puede haber dos grados de igual nombre.

- **actualizar:** Este método recibe un objeto Grado que se supone que ha sido actualizado por medio de sus setters, pero en el almacenamiento está aún la información antigua. Este método actualiza el grado en el sistema de almacenamiento.

```
1. public boolean actualizar(Grado g) {
2.     return this.gradosRepository.actualizar(g);
3. }
```

En este caso el Service no tiene que hacer ninguna comprobación previa sobre el grado y simplemente se limita a ordenarle al Repository que lo actualice en la base de datos. Este patrón, donde un método se limita a llamar al mismo método que tiene otro objeto es muy frecuente, y se llama “delegación”, pero no significa en ningún caso, que el Service “sea lo mismo” que el Repository.

- **borrar:** Borra del sistema de almacenamiento el grado recibido como parámetro.

```
1. public boolean borrar(Grado g) {
2.     return this.gradosRepository.borrar(g);
3. }
```

Al igual que en el caso anterior, el Service delega directamente en el Repository sin necesidad de realizar ninguna comprobación adicional.

- **nuevaAsignatura:** Este método añade al grado pasado como parámetro una asignatura cuyos datos se pasan como parámetro, siempre que no exista en el grado una asignatura con ese nombre (en cuyo caso, se lanzará una `IllegalArgumentException`). Si todo va bien, se creará la asignatura en el almacenamiento y se añadirá a las asignaturas del grado. Se devolverá la asignatura creada.

```
1. public Asignatura nuevaAsignatura(Grado g, String n, int nc, int cr) {
2.     for (Asignatura a : g.getAsignaturas()) {
3.         if (a.getNombre().equals(n)) {
4.             throw new IllegalArgumentException("Ya existe la asignatura "+n+" en el grado "+g.getNombre());
5.         }
6.     }
7.     Asignatura a = this.gradosRepository.nuevaAsignatura(g, n, nc, cr);
8.     if (a != null) {
9.         g.getAsignaturas().add(a);
10.    }
11.    return a;
12. }
```

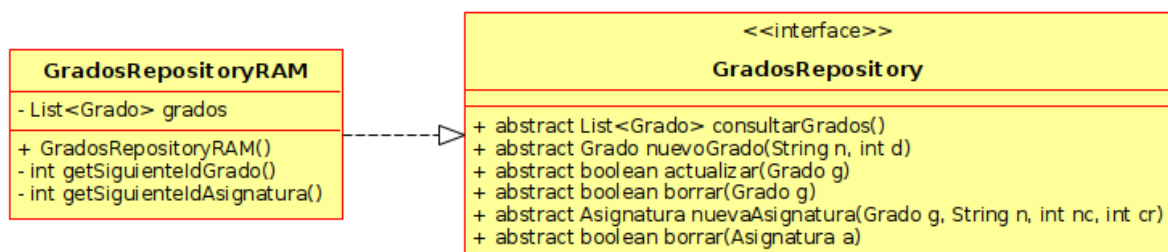
- **borrar:** Borra del almacenamiento y de la lista de asignaturas del grado correspondiente, la asignatura pasada como parámetro.

```
1. public boolean borrar(Asignatura a) {
2.     boolean r=true;
3.     if (this.gradosRepository.borrar(a)) {
4.         a.getGrado().getAsignaturas().remove(a);
5.     } else {
6.         r=false;
7.     }
8.     return r;
9. }
```

7.8.- Test de la lógica de negocio

Llegados a este punto es el momento de realizar test para ver si todo va bien. El problema es que los datos están en una base de datos, que aún no está disponible, y aunque lo estuviera, los tests dependerían del estado de la base de datos y no serían predecibles. Afortunadamente, la arquitectura hexagonal hace que podamos añadir al sistema una clase que “simule” en memoria ram el comportamiento del sistema de almacenamiento, que recordemos, es una interfaz.

Podemos, por tanto, crear el paquete **dam.grados.infraestructura.ram** y añadir una clase llamada **GradosRepositoryRAM** que se utilice para hacer tests del resto de clases que necesiten acceder al almacenamiento (como **GradosServiceImpl**). La clase **GradosRepositoryRAM** simplemente mantendrá en memoria un **ArrayList<Grado>** donde se almacenará la información. Esa clase necesitará métodos privados que nos den el id del siguiente grado y asignatura que se vayan a crear.



```
1. public class GradosRepositoryRAM implements GradosRepository {
2.     private List<Grado> grados;
3.     public GradosRepositoryRAM(){
4.         this.grados=new ArrayList<>();
5.     }
6.     @Override
7.     public List<Grado> consultarGrados() { return this.grados; }
8.     private int getSiguienteIdGrado(){
9.         int id=0;
10.        for(Grado g:grados){
11.            if(g.getId()>id){
12.                id=g.getId()+1;
13.            }
14.        }
15.        return id;
16.    }
17.    private int getSiguienteIdAsignatura(){
18.        int id=0;
19.        for(Grado g:grados){
20.            for(Asignatura a:g.getAsignaturas()) {
21.                if (a.getId() > id) {
22.                    id = a.getId() + 1;
23.                }
24.            }
25.        }
26.        return id;
27.    }
28.    @Override
29.    public Grado nuevoGrado(String n, int d) {
30.        Grado g = new Grado(this.getSiguienteIdGrado(),n,d);
31.        this.grados.add(g);
32.        return g;
33.    }
34.    @Override
35.    public boolean actualizar(Grado g) {
36.        int posicion = this.grados.indexOf(g);
37.        this.grados.set(posicion,g);
38.        return true;
39.    }
```

```

40.     @Override
41.     public boolean borrar(Grado g) {
42.         this.grados.remove(g);
43.         return true;
44.     }
45.     @Override
46.     public Asignatura nuevaAsignatura(Grado g, String n, int nc, int cr) {
47.         return new Asignatura(g, this.getSiguienteIdAsignatura(), n, nc, cr);
48.     }
49.     @Override
50.     public boolean borrar(Asignatura a) {
51.         Grado g = a.getGrado();
52.         return g.getAsignaturas().contains(a);
53.     }
54. }

```

Para hacer los tests de **GradosServiceImpl**, tendremos que hacer algunas modificaciones respecto a lo que estamos acostumbrados.

- Añadiremos una variable de instancia llamada **gradosService**, que será el objeto que usaremos en los tests
- Añadiremos un método **inicializarRepository** que etiquetaremos con **@Before**. Este método se ejecutará siempre antes de cada test, y allí crearemos una base de datos limpia, con varios datos de ejemplo, y haremos que el **gradosService** la use. De esta forma, cada test será independiente del anterior.

```

1. public class GradosServiceImplTest {
2.     private GradosService gradosService;
3.     @Before
4.     public void inicializarRepository() throws Exception {
5.         GradosRepository gr = new GradosRepositoryRAM();
6.         Grado mates = gr.nuevoGrado("Matemáticas",4);
7.         Grado infor = gr.nuevoGrado("Informática",3);
8.         gr.nuevaAsignatura(mates,"Álgebra",1,60);
9.         gr.nuevaAsignatura(mates,"Geometría",1,60);
10.        gr.nuevaAsignatura(infor,"Programación",1,30);
11.        gr.nuevaAsignatura(infor,"Arquitectura de computadores",1,30);
12.        this.gradosService = new GradosServiceImpl(gr);
13.    }
14. }

```

Hecho esto, ya podemos programar todos los tests que queramos, de la forma habitual, como por ejemplo estos:

- Añadir un grado llamado Matemáticas y comprobar que salta una **IllegalArgumentException**

```

1. @Test(expected = IllegalArgumentException.class)
2. public void test1() {
3.     Grado g = gradosService.nuevoGrado("Matemáticas",4);
4.     fail("No se ha lanzado la excepción que se esperaba");
5. }

```

- Consultar todos los grados y comprobar que hay dos y que el segundo se llama Informática

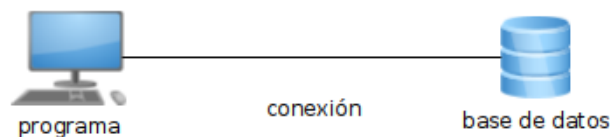
```

1. @Test
2. public void test2() {
3.     List<Grado> grados = gradosService.consultarGrados();
4.     assertEquals(2, grados.size());
5.     assertEquals("Informática",grados.get(1).getNombre());
6. }

```

7.9.- Implementación del acceso a base de datos

La aplicación que estamos haciendo sigue el modelo tradicional en el que el programa se conecta directamente al servidor donde está la base de datos, y le lanza sentencias SQL para manipular la información. Este ha sido el enfoque utilizado por las aplicaciones de escritorio durante muchos años, y por ese motivo, es el que vamos a estudiar aquí.



La base de datos se encuentra en un servidor MariaDB que contiene las siguientes tablas:

GRADOS (id, nombre, duracion)

ASIGNATURAS (id, nombre, grado_id, curso, creditos)

Para que nuestro programa se conecte a la base de datos y trabaje con ella usaremos la librería **JDBC (Java DataBase Connectivity)**. Esta librería permite establecer una conexión con una base de datos relacional, lanzarle cualquier sentencia SQL y procesar su respuesta. Hoy día esta librería se considera “de bajo nivel” puesto que trabaja directamente con las tablas (no hay objetos) y debemos lanzar sentencias SQL con las operaciones que graben y recuperen nuestros objetos de las tablas. Por tanto, tendremos que implementar dos de las interfaces de nuestra arquitectura:

- OrigenDatos → Haremos una clase llamada **BaseDatosJDBC** que la implementará, y permitirá conectarnos y desconectarnos a la base de datos MariaDB (el proveedor se puede cambiar fácilmente por otro)
- GradosRepository → Haremos una clase llamada **GradosRepositoryJDBC** que la implementará, y permitirá grabar objetos Grado y Asignatura en la base de datos MariaDB.

Crearemos un paquete llamado **dam.grados.infraestructure.jdbc** y en él programaremos dichas clases.

7.9.1.- Conexión y desconexión de la base de datos

La primera clase que haremos será **BaseDatosJDBC**, que representa una base de datos relacional a la que nos conectaremos con la librería JDBC. Dicha clase tendrá como variable de instancia un objeto **Connection** (del paquete **java.sql**) que almacena la conexión con la base de datos. La clase implementará **OrigenDatos**, lo que permitirá obtener y cerrar una conexión con la base de datos.

```

1. public class BaseDatosJDBC implements OrigenDatos {
2.     private Connection conexion;
3.     public Connection getConexion() {
4.         return this.conexion;
5.     }
6.     @Override
7.     public boolean conectar() {
8.         // por hacer
9.     }
10.    @Override
11.    public boolean desconectar() {
12.        // por hacer
13.    }
14. }

```

Para conectarnos con JDBC a una base de datos, lo primero que tenemos que hacer es agregar al archivo **build.gradle.kts** la librería que permite trabajar con el modelo concreto de base de datos usada, que en nuestro caso es MariaDB.

`implementation("org.mariadb.jdbc:mariadb-java-client:3.5.3")`

A continuación, dentro del método **conectar** debermos llamar al método **DriverManager.getConnection** para conectarnos a la base de datos y obtener un objeto **Connection** que representa la conexión obtenida. Dicho método necesita tres parámetros:

- **Cadena de conexión:** Es un String que contiene la descripción y la ubicación de la base de datos a la que nos vamos a conectar. Cada fabricante de bases de datos tiene su propio formato para la cadena de conexión. En el caso de MariaDB, dicho formato es:

"jdbc:mariadb://127.0.0.1:3306/grados"
ip del puerto base de
servidor datos

- **Usuario:** Es el nombre de usuario con el que nos conectaremos a la base de datos. Lógicamente, el administrador de la base de datos nos lo ha tenido que crear y habernos dado los permisos que vamos a necesitar en nuestro programa
- **Clave:** La contraseña del usuario que usamos para conectarnos a la base de datos

Programaremos el método **conectar** de esta forma:

```

1. public boolean conectar() {
2.     boolean r=true;
3.     try{
4.         String cc= "jdbc:mariadb://127.0.0.1:3306/grados";
5.         String usuario="java";
6.         String clave="java";
7.         this.conexion = DriverManager.getConnection(cc,usuario,clave);
8.     }catch(SQLException e){
9.         System.out.println("Error al conectar: "+e.getMessage());
10.        r=false;
11.    }
12.    return r;
13. }

```

Para desconectarnos de la base de datos, llamaremos al método **close** de la conexión

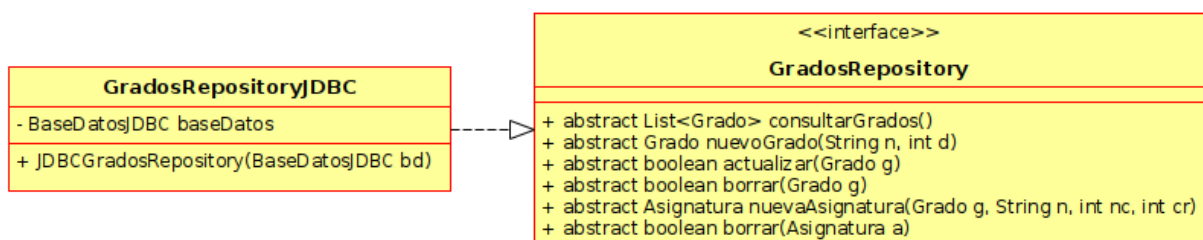

```

1. public boolean desconectar() {
2.     boolean r=true;
3.     try{
4.         this.conexion.close();
5.     }catch(SQLException e){
6.         System.out.println("Error al desconectar: "+e.getMessage());
7.         r=false;
8.     }
9.     return r;
10. }

```

7.9.2.- Realización de consultas

Vamos a comenzar la programación de la clase **GradosRepositoryJDBC**. Esta clase tendrá como variable de instancia un objeto **BaseDatosJDBC** que recibirá en su constructor.



```

1. public class GradosRepositoryJDBC implements GradosRepository {
2.     private BaseDatosJDBC baseDatos;
3.     public GradosRepositoryJDBC(BaseDatosJDBC baseDatos){
4.         this.baseDatos=baseDatos;
5.     }
6. }

```

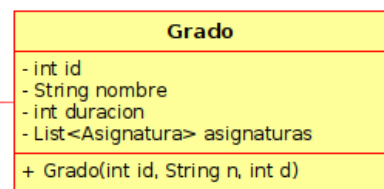
A continuación, programaremos el método **consultarGrados**. Este método deberá consultar todos los objetos **Grado** que hay en la base de datos y devolverlos en una lista.

Como la base de datos es relacional, está formada por tablas y en la tabla **grados** encontramos la información que necesitamos para obtener los objetos. De hecho, podemos identificar cada fila de la tabla con un objeto **Grado**

```

MariaDB [grados]> select * from grados;
+----+-----+-----+
| id | nombre                | duracion |
+----+-----+-----+
| 1  | Ingeniería Informática | 4        |
| 2  | Medicina               | 6        |
| 3  | Derecho                | 5        |
+----+-----+-----+
3 rows in set (0,002 sec)

```



La idea es lanzar la sentencia **select * from grados** a la base de datos y recorrer la tabla que se obtiene como resultado. Por cada fila, crearemos un objeto **Grado**, que añadiremos a un **List<Grado>** que habremos creado inicialmente, y que devolveremos al terminar el método.

Los pasos para hacer esto son:

- Creamos un String con la sentencia SQL que vamos a lanzar a la base de datos

```
String sql="select * from grados";
```

- Llamamos al método **prepareStatement** del objeto **Connection**, pasándole el String con la sentencia SQL y obtendremos un **PreparedStatement**

```
PreparedStatement s = this.baseDatos.getConnection().prepareStatement(sql);
```

El **PreparedStatement** es un objeto que representa una sentencia SQL

- Usamos el método **executeQuery** del **PreparedStatement** y con ello, la sentencia SQL se lanzará y como es una consulta, se obtendrá como resultado un objeto **ResultSet**

```
ResultSet r = s.executeQuery();
```

El **ResultSet** es un objeto que representa la tabla que se obtiene tras hacer una consulta SQL

- Usamos el método **next()** del **ResultSet** dentro de un bucle while. Esto creará un bucle donde que en cada iteración estaremos trabajando sobre una fila diferente del **ResultSet**

```
while(r.next()){  
    // en cada iteración el ResultSet está posicionado en una fila diferente  
}
```

En cada iteración de ese bucle, el cursor interno del **ResultSet** apunta a una fila diferente.

- Usamos los métodos **getString**, **getInt**, **getDouble**, **getBoolean**, etc del **ResultSet** para obtener el valor de la columna que buscamos, con el tipo de dato que indica el nombre del método. Por ejemplo, si queremos recuperar el id del grado, que es un int, usaremos **getInt**, y si queremos recuperar el nombre del grado, que es un String, usaremos **getString**. Estos métodos reciben como parámetro el nombre de la columna que se desea recuperar.

```
while(r.next()){ // recorro todas las filas del ResultSet  
    int id=r.getInt("id"); // recupero el id del grado  
    String nombre=r.getString("nombre"); // recupero el nombre del grado  
    int duracion=r.getInt("duracion"); // recupero la duración del grado  
    Grado g = new Grado(id,nombre,duracion); // creo un grado con los datos de la fila recorrida  
    lista.add(g); // añado el grado a la lista de grados que hay que devolver  
}
```

Los métodos **getString**, **getInt**, **getDouble**, etc permiten recuperar el valor de una columna, dentro de la fila en la que esté posicionado el **ResultSet**

Lo único que queda destacar, es que todos los métodos que trabajan con bases de datos pueden lanzar una **SQLException** (por ejemplo, si se cae la conexión al servidor, o si la sentencia SQL que se lanza es incorrecta). Por ese motivo, todo el código del método estará encerrado en un try-catch.

```

1. public List<Grado> consultarGrados() {
2.     List<Grado> lista = new ArrayList<>();
3.     try {
4.         String sql="select * from grados";
5.         PreparedStatement s = this.baseDatos.getConexion().prepareStatement(sql);
6.         ResultSet r = s.executeQuery();
7.         while(r.next()){ // recorro todas las filas del ResultSet
8.             int id=r.getInt("id"); // recupero el id del grado
9.             String nombre=r.getString("nombre"); // recupero el nombre del grado
10.            int duracion=r.getInt("duracion"); // recupero la duración del grado
11.            Grado g = new Grado(id,nombre,duracion); // creo un grado con los datos de la fila recorrida
12.            lista.add(g); // añado el grado a la lista de grados que hay que devolver
13.        }
14.    }catch(SQLException e){
15.        System.out.println("Error al consultar grados: "+e.getMessage());
16.    }
17.    return lista;
18. }

```

7.9.3.- Realización de consultas con variables

El método que consulta los grados está muy bien, pero tiene un problema. Cada objeto **Grado** creado contiene una lista de objetos **Asignatura**, que se ha quedado vacía. Por tanto, es necesario, tras crear un objeto **Grado**, añadirle todas sus asignaturas, y para eso, será necesario volver a consultar la base de datos.

La sentencia SQL que permite consultar las asignaturas de un grado es:

*select * from asignaturas where grado_id=id del grado*

Como vemos, esta sentencia no es “constante”, sino que posee un dato (el id del grado) que debe ser proporcionado por el programa. Lo normal es tener siempre ese tipo de valores guardados en variables, o disponer de un objeto que nos lo de por medio de un método.

Para no enmarañar el código del método **consultarGrados**, vamos a hacer un método privado auxiliar llamado **añadirAsignaturas**, que reciba como parámetro un **Grado** y se encargue de consultar los objetos **Asignatura** de la base de datos, y añadirlos a dicho grado. Este método privado será llamado dentro de **consultarGrados**, justo tras crear el objeto Grado.

```

1. public List<Grado> consultarGrados() {
2.     List<Grado> lista = new ArrayList<>();
3.     try {
4.         String sql="select * from grados";
5.         PreparedStatement s = this.baseDatos.getConexion().prepareStatement(sql);
6.         ResultSet r = s.executeQuery();
7.         while(r.next()){
8.             int id=r.getInt("id");
9.             String nombre=r.getString("nombre");
10.            int duracion=r.getInt("duracion");
11.            Grado g = new Grado(id,nombre,duracion);
12.            this.añadirAsignaturas(g); // añado al grado sus asignaturas
13.            lista.add(g);
14.        }
15.    }catch(SQLException e){ System.out.println("Error al consultar grados: "+e.getMessage()); }
16.    return lista;
17. }
18. }
19. private void añadirAsignaturas(Grado grado) {
20.     // consulto las asignaturas del grado, y se las añado
21. }

```

Dentro de **añadirAsignaturas** lanzaremos la sentencia SQL que consulta las asignaturas de un grado, recorreremos la tabla de resultados, por cada fila crearemos un objeto **Asignatura** y se lo añadiremos a la lista de asignaturas del objeto **Grado** recibido como parámetro.

Los pasos son:

- Creamos un String con la sentencia SQL que vamos a lanzar a la base de datos. Como intervienen variables, pondremos un interrogante **?** en cada dato que deba ser sustituido por una variable

```
String sql="select * from asignaturas where grado_id = ?";
```

Los datos de la sentencia que deban ser sustituidos por variables se ponen con un **?**

- Llamamos al método **prepareStatement** del objeto **Connection**, pasándole el String con la sentencia SQL y obtendremos un **PreparedStatement**

```
PreparedStatement s = this.baseDatos.getConnection().prepareStatement(sql);
```

- Usamos los métodos **setString**, **setInt**, **setDouble**, **setBoolean**, etc para sustituir cada interrogante de la sentencia por una variable. Dichos métodos reciben el número del interrogante (empezando en 1) y el dato por el que se sustituye dicho interrogante.

```
s.setInt(1,grado.getId()); // sustituye el primer interrogante por grado.getId()
```

Los métodos **setString**, **setInt**, etc permiten sustituir los interrogantes por variables

- La consulta se lanza y se procesa de la misma forma que antes. En esta ocasión al recorrer el **ResultSet** se crea un objeto **Asignatura** con los datos de cada fila y se añade dicha asignatura al objeto **Grado** que se ha pasado como parámetro.

```
1. private void añadirAsignaturas(Grado grado) {
2.     try {
3.         String sql="select * from asignaturas where grado_id=?";
4.         PreparedStatement s = this.baseDatos.getConnection().prepareStatement(sql);
5.         s.setInt(1,grado.getId()); // sustituye el primer interrogante por grado.getId()
6.         ResultSet r = s.executeQuery();
7.         while(r.next()){
8.             int id=r.getInt("id");
9.             String nombre=r.getString("nombre");
10.            int curso=r.getInt("curso");
11.            int creditos=r.getInt("creditos");
12.            Asignatura a = new Asignatura(grado,id,nombre,curso,creditos);
13.            grado.getAsignaturas().add(a);
14.        }
15.    }catch(SQLException e){
16.        System.out.println("Error al consultar asignaturas: "+e.getMessage());
17.    }
18. }
```

Es posible hacer este tipo de consultas sin usar los interrogantes, colocando directamente en el String las variables, pero eso, además de ser muy engorroso, es propenso a ataques de SQL-Injection. Usar los interrogantes nos protege contra ese ataque.

7.9.4.- Inserción de datos

El método **nuevoGrado** nos permite registrar un nuevo grado en la base de datos, lo que significa que deberá grabarse una fila en la tabla **grados** con la información correspondiente a dicho grado (las asignaturas se insertarán posteriormente, una vez que el grado ha sido creado). Además, dicho método deberá devolver un objeto **Grado** con la información del grado que ha sido insertado.

La sentencia SQL que permite insertar un grado es:

```
insert into grados(nombre,duracion) values(nombre del grado, duración del grado)
```

Los pasos para realizar la inserción serán:

- Creamos un String con la sentencia que vamos a lanzar a la base de datos. Como intervienen variables, pondremos un interrogante **?** en cada dato que deba ser sustituido por una variable

```
String sql="insert into grados(nombre,duracion) values (?,?)";
```

- Llamamos al método **prepareStatement** del objeto **Connection**, pasándole el String con la sentencia SQL y obtendremos un **PreparedStatement**

```
PreparedStatement s = this.baseDatos.getConnection().prepareStatement(sql);
```

- Usamos los métodos **setString**, **setInt**, **setDouble**, **setBoolean**, etc para sustituir cada interrogante de la sentencia por una variable. Dichos métodos reciben el número del interrogante (empezando en 1) y el dato por el que se sustituye dicho interrogante.

```
s.setString(1,n); // sustituye el primer ? por la variable "n", que es el primer parámetro  
s.setInt(2,d); // sustituye el segundo ? por la variable "d", que es el segundo parámetro
```

- Como la sentencia es un INSERT, no usaremos **executeQuery**, sino que en su lugar, llamaremos a **executeUpdate**. Este método realiza la inserción de la fila en la base de datos.

```
s.executeUpdate();
```

El método **executeUpdate** permite lanzar cualquier sentencia que no sea una consulta

- Por último, necesitamos crear un objeto **Grado** con el grado que ha sido creado y devolverlo. Tenemos el nombre y duración de ese grado, pero nos falta el id del grado, y en el próximo punto lanzaremos una sentencia SQL que nos permitirá obtenerlo. De momento, crearemos un método auxiliar en la clase **BaseDatosJDBC** que nos lo devuelva y dejaremos indicada su llamada.

```
int id=this.baseDatos.getUltimoIdGenerado(s);  
g=new Grado(id,n,d);
```

```

1. public Grado nuevoGrado(String n, int d) {
2.     Grado g = null;
3.     try {
4.         String sql="insert into grados(nombre,duracion) values(?,?)";
5.         PreparedStatement s = this.baseDatos.getConnection().prepareStatement(sql);
6.         s.setString(1,n); // sustituye el primer ? por la variable "n"
7.         s.setInt(2,d); // sustituye el segundo ? por la variable "d"
8.         s.executeUpdate();
9.         int id=this.baseDatos.getUltimoIdGenerado(s);
10.        g=new Grado(id,n,d);
11.    }catch(SQLException e){
12.        System.out.println("Error al crear el grado: "+e.getMessage());
13.    }
14.    return g;
15. }

```

7.9.5.- Consulta del último ID generado

Cuando hacemos un insert en el que hay un campo autoincrementado, la base de datos se encarga de generarle su id, y en principio, no hay forma de poder saber cuál ha sido ese id. Afortunadamente, JDBC nos da una forma de obtenerlo tras realizar la inserción. Como ese procedimiento es común para todas las bases de datos JDBC, lo ideal es hacer un método en la clase **BaseDatosJDBC** que nos lo devuelva.

BaseDatosJDBC
- Connection conexion
+ BaseDatosJDBC() + Connection getConnection() + int getUltimoIdGenerado(PreparedStatement ps) throws SQLException

El método **getUltimoIdGenerado** recibe un **PreparedStatement** que representa una sentencia INSERT que acaba de ser lanzada y nos devolverá el id generado. Para ello, bastará con llamar al método **getGeneratedKeys** del **PreparedStatement** y se obtendrá un **ResultSet** cuya primera fila es el valor que buscamos. Por tanto, se programará así:

```

1. public int getUltimoIdGenerado(PreparedStatement ps) throws SQLException{
2.     int id=0;
3.     ResultSet r = ps.getGeneratedKeys();
4.     while(r.next()){
5.         id=r.getInt(1);
6.     }
7.     return id;
8. }

```

Para que el método **getGeneratedKeys** funcione, el **PreparedStatement** debe haber sido creado con la constante **PreparedStatement.RETURN_GENERATED_KEYS** cuando se llama al método **prepareStatement**.

Por tanto, hay que modificar el método **nuevoGrado** de la clase **GradosRepositoryJDBC** para obtener el **PreparedStatement** de esta forma:

```

PreparedStatement s =
this.baseDatos.getConnection().prepareStatement(sql, PreparedStatement.RETURN_GENERATED_KEYS);

```

De forma muy similar, podemos programar ya el método **nuevaAsignatura**, que graba en la base de datos una asignatura asociada a un grado y devuelve un objeto **Asignatura** con su información.

```
1. public Asignatura nuevaAsignatura(Grado g, String n, int nc, int cr) {
2.     Asignatura a = null;
3.     try {
4.         String sql="insert into asignaturas(nombre,grado_id,curso,creditos) values(?,?,?,?)";
5.         PreparedStatement s =
6.             this.baseDatos.getConnection().prepareStatement(sql,PreparedStatement.RETURN_GENERATED_KEYS);
7.         s.setString(1,n);
8.         s.setInt(2,g.getId());
9.         s.setInt(3,nc);
10.        s.setInt(4,cr);
11.        s.executeUpdate();
12.        int id=this.baseDatos.getUltimoIdGenerado(s);
13.        a=new Asignatura(g,id,n,nc,cr);
14.    }catch(SQLException e){
15.        System.out.println("Error al crear la asignatura: "+e.getMessage());
16.    }
17.    return a;
18. }
```

7.9.7.- Actualización de datos

La actualización de datos es muy similar a la inserción, ya que el procedimiento es exactamente igual (rellenar el String sql y lanzarlo con **executeUpdate**). Lo único que cambiará es la sentencia SQL, que en esta ocasión será un UPDATE.

Para actualizar un grado tendremos el método **actualizar**. Dicho método recibe un objeto **Grado** que se supone, ha sido modificado por medio de sus setters, pero aún no ha sido guardado en la base de datos. El método cogerá sus datos actualizados (llamando a sus getters) y lanzará la sentencia anterior en la base de datos.

El procedimiento es exactamente igual al de inserción, pero cambiando la sentencia SQL

```
1. public boolean actualizar(Grado g) {
2.     boolean r=true;
3.     try{
4.         String sql="update grados set nombre=?, duracion=? where id=?";
5.         PreparedStatement s = this.baseDatos.getConnection().prepareStatement(sql);
6.         s.setString(1,g.getNombre());
7.         s.setInt(2,g.getDuracion());
8.         s.setInt(3,g.getId());
9.         s.executeUpdate();
10.    }catch(SQLException e){
11.        System.out.println("Error al actualizar: "+e.getMessage());
12.        r=false;
13.    }
14.    return r;
15. }
```

7.9.8.- Borrado de datos

El borrado de datos es exactamente igual a la inserción y actualización. Lo único que cambia es la sentencia SQL, que ahora será de tipo DELETE.

El método **borrar(Grado)** elimina un **Grado** de la base de datos y sus asignaturas asociadas. Como no siempre la base de datos tiene configurado el DELETE CASCADE, vamos a asegurarnos de que todo se borra correctamente lanzando dos sentencias DELETE, uno para borrar las asignaturas vinculadas al grado que queremos borrar, y otra para borrar al grado en si.

```
1. public boolean borrar(Grado g) {
2.     boolean r=true;
3.     try{
4.         // eliminamos las asignaturas del grado "g"
5.         String sql = "delete from asignaturas where grado_id=?";
6.         PreparedStatement s = this.baseDatos.getConnection().prepareStatement(sql);
7.         s.setInt(1,g.getId());
8.         s.executeUpdate();
9.         // eliminamos el grado "g"
10.        sql="delete from grados where id=?";
11.        s = this.baseDatos.getConnection().prepareStatement(sql);
12.        s.setInt(1,g.getId());
13.        s.executeUpdate();
14.    }catch(SQLException e){
15.        System.out.println("Error al borrar: "+e.getMessage());
16.        r=false;
17.    }
18.    return r;
}
```

De forma similar, podemos programar el método **borrar(Asignatura)**, que borra de la base de datos la asignatura que se pasa como parámetro.

```
1. public boolean borrar(Asignatura a) {
2.     boolean r=true;
3.     try{
4.         String sql="delete from asignaturas where id=?";
5.         PreparedStatement s = this.baseDatos.getConnection().prepareStatement(sql);
6.         s.setInt(1,a.getId());
7.         s.executeUpdate();
8.     }catch(SQLException e){
9.         System.out.println("Error al borrar: "+e.getMessage());
10.        r=false;
11.    }
12.    return r;
13. }
```

7.9.9.- Transacciones

El método **borrar(Grado)** lanza dos sentencias seguidas de borrado a la base de datos y eso tiene el inconveniente de que si se cae el sistema entre ellas, puede ocurrir que una se lance y la otra no, dejando la operación a la mitad.

En general, no es buena idea que las operaciones sobre la base de datos que incluyen muchas sentencias se puedan quedar a medias, porque se puede perder información o la base de datos quedar en un estado inconsistente. Para evitarlo, las bases de datos soportan las **transacciones**, que son intervalos de tiempo donde un usuario trabaja en un entorno aislado, y todo lo que hace no es definitivo hasta que la transacción se confirma (**commit**). Si el sistema se cae durante la transacción, o el usuario decide deshacer la transacción (**rollback**), todo se queda como estaba al inicio de esta.

Para iniciar la transacción se usa el método **setAutoCommit(false)** del objeto **Connection**, y si las dos operaciones de borrado se ejecutan correctamente, se confirmará la transacción (**commit**)

Modificamos el método **borrar(Grado)** para que las dos sentencias se hagan en una transacción, de esta forma:

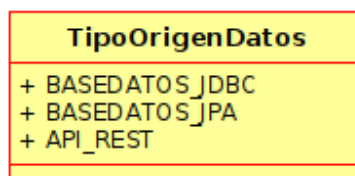
```
1. public boolean borrar(Grado g) {
2.     boolean r=true;
3.     try{
4.         baseDatos.getConnection().setAutoCommit(false); // iniciamos la transacción
5.         String sql = "delete from asignaturas where grado_id=?";
6.         PreparedStatement s = this.baseDatos.getConnection().prepareStatement(sql);
7.         s.setInt(1,g.getId());
8.         s.executeUpdate();
9.         sql="delete from grados where id=?";
10.        s = this.baseDatos.getConnection().prepareStatement(sql);
11.        s.setInt(1,g.getId());
12.        s.executeUpdate();
13.        baseDatos.getConnection().commit(); // confirmamos la transacción
14.        baseDatos.getConnection().setAutoCommit(true); // seguimos trabajando en modo normal (sin transacción)
15.    }catch(SQLException e){
16.        System.out.println("Error al borrar: "+e.getMessage());
17.        r=false;
18.    }
19.    return r;
20. }
```

7.10.- Contexto de la aplicación

La aplicación que hemos diseñado trabaja con un Service y un Repository, que junto con el OrigenDatos, son interfaces cuya implementación JDBC ya hemos programado.

Llamaremos **contexto** de la aplicación a las clases concretas que elegimos en nuestro programa para implementar todas esas interfaces. Dicho contexto va a ser una clase **singleton**, es decir, estará diseñada de forma que solo pueda crearse un objeto **Contexto** en la memoria. Dicho objeto contendrá las instancias del Service, Repository y OrigenDatos que serán usadas por la aplicación durante su funcionamiento.

Comenzaremos programando una enum **TipoOrigenDatos** con los posibles orígenes de datos que necesitaremos. En este tema el origen de datos (el lugar donde están los grados con sus asignaturas) es una base de datos JDBC, pero en el próximo tema, los obtendremos consultando un API Rest y además cambiaremos la librería JDBC por la librería JPA. Por tanto, la enum será esta:



```
1. public enum TipoOrigenDatos {
2.     BASEDATOS_JDBC, BASEDATOS_JPA, REST_API;
3. }
```

A continuación, diseñaremos la clase **Contexto**, de forma que solo pueda haber un único objeto de esa clase en la memoria. El truco está en hacer su constructor privado, y guardar en una propiedad estática el único objeto de la clase que podrá existir. Dicho objeto se recupera por medio de un método estático **getInstancia**, que crea el método **inicializar**.

Context
- static Context INSTANCIA - OrigenDatos origenDatos - GradosRepository repository - GradosService service
- Context() + GradosRepository getGradosRepository() + GradosService getGradosService() + void cerrar() + static void inicializar(TipoOrigenDatos tipoOrigen) + static Context getInstancia()

La descripción de los métodos es:

- **Constructor:** No hace nada, y lo único que hace al ser privado es impedir su uso desde fuera de la clase.
- **Getters:** Permiten obtener las instancias del Service y Repository usados
- **cerrar:** Se desconecta del origen de datos utilizado
- **inicializar:** Permite inicializar el OrigenDatos, Repository y Service de acuerdo al tipo de origen pasado como parámetro
- **getInstancia:** Devuelve la propiedad estática **INSTANCIA**

La programación de la clase, según la descripción de sus métodos es:

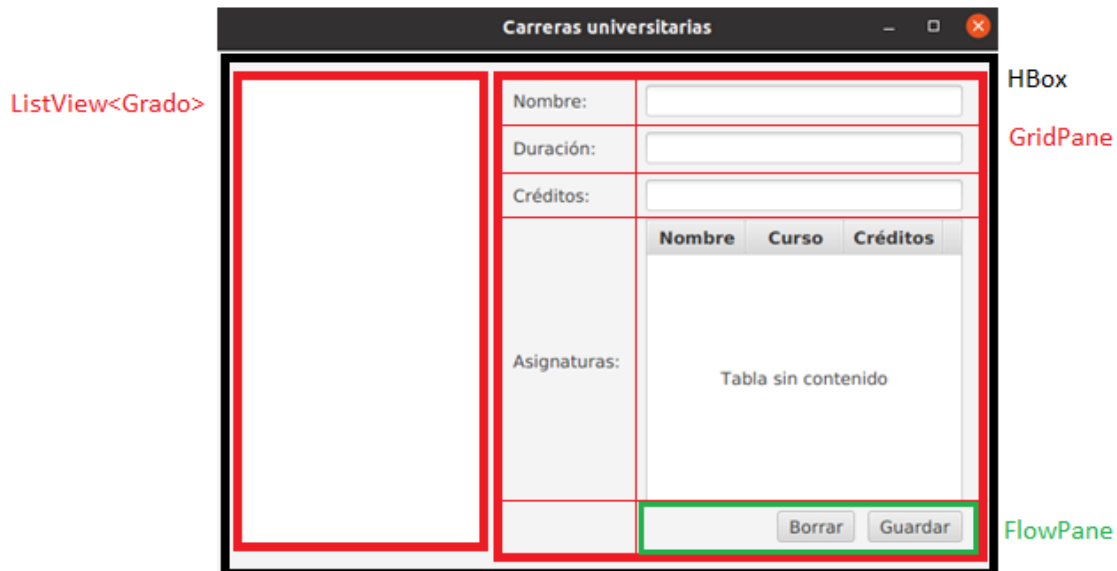
```

1. public class Context {
2.     private static Context INSTANCIA;
3.     private OrigenDatos origenDatos;
4.     private GradosRepository repository;
5.     private GradosService service;
6.     private Context(){ }
7.     public static Context getInstancia(){
8.         if(Context.INSTANCIA==null){
9.             throw new IllegalStateException("No se ha inicializado el context");
10.        }
11.        return Context.INSTANCIA;
12.    }
13.    public static void inicializar(TipoOrigenDatos tipoOrigen){
14.        if(Context.INSTANCIA!=null){
15.            throw new IllegalStateException("Ya se ha inializado el context");
16.        }
17.        INSTANCIA=new Context();
18.        if(tipoOrigen.equals(TipoOrigenDatos.BASEDATOS_JDBC)){
19.            BaseDatosJDBC baseDatos=new BaseDatosJDBC();
20.            if(baseDatos.conectar()) {
21.                INSTANCIA.origenDatos = baseDatos;
22.                INSTANCIA.repository = new GradosRepositoryJDBC(baseDatos);
23.            }else{
24.                throw new IllegalStateException("No se pudo conectar con la base de datos");
25.            }
26.        }else{
27.            throw new IllegalStateException("Tipo de origen de datos no admitido");
28.        }
29.        INSTANCIA.service=new GradosServiceImpl(INSTANCIA.repository);
30.    }
31.    public GradosService getGradosService() { return this.service; }
32.    public void cerrar(){
33.        if(!Context.INSTANCIA.origenDatos.desconectar()){
34.            throw new IllegalStateException("No se pudo desconectar del origen de datos");
35.        }
36.    }
37. }

```

7.10.- Diseño de las vistas

Esta aplicación tiene dos vistas que llamaremos **frmGrados.fxml** y **frmNuevaAsignatura.fxml**. Usaremos SceneBuilder para dibujar ambas, de forma que tengan el aspecto que hemos mostrado en las páginas anteriores. Aquí vemos la división en paneles de **frmGrados**:



Los pasos que hay que seguir para diseñar **frmGrados** son:

- Comenzamos añadiendo un **HBox**, lo seleccionamos y en la pestaña **Layout** ponemos en el primer cuadro de **padding** el valor 20. Pulsamos el botón > para que ese valor se propague al resto de direcciones.
- Le añadimos al **HBox** un **ListView** y le pondremos como id: **IstGradosUniversitarios**
- Añadimos al **HBox** un **GridPane**, lo seleccionamos y en **layout** buscamos **Hgrow** y le ponemos **ALWAYS**. También buscamos **margin** y le ponemos por la izquierda 20 y por la derecha 10
- Pulsamos el botón derecho en alguna fila del **GridPane** y seleccionamos **add row above**. Insertamos dos filas más a las que ya nos vienen por defecto. En total, el **GridPane** deberá tener 5 filas y 2 columnas.
- Para las primeras 3 filas arrastramos **Label** a su primera columna y les ponemos los letreros "Nombre", "Duración" y "Créditos". Arrastramos en la segunda columna **TextField** y les ponemos de id **txtNombre**, **txtDuracion** y **txtCreditos**. A este último le pondremos **disabled**.
- Para la 4ª fila, arrastramos en su primera columna y **Label** con el letrero "Asignaturas" y en la segunda columna un **TableView**, cuyo id será **tabAsignaturas**
- En la sección **controls** buscamos un **TableColumn** y lo añadimos dentro del **TableView**, de forma que la tabla tenga 3 columnas.
- Seleccionamos los objetos **TableColumn** que forman las columnas del **TableView** y les cambiamos el nombre para ponerles "Nombre", "Curso" y "Créditos". Les pondremos como id **colNombre**, **colCurso** y **colCreditos**.
- Movemos las guías que marcan las columnas del **GridPane** de forma que los **Label** tengan su tamaño exacto y los **TextField** se hagan más grandes, ocupando casi toda la fila.

- Pulsando **CTRL** vamos pulsando en los números 0,1,2 y 4 que tienen las filas del **GridPane** de forma que se vayan poniendo en amarillo. Una vez que están todos seleccionados, nos vamos a la sección **Layout** y donde pone **percent height** escribimos 10. De esa forma, haremos que esas filas tengan una altura del 10% de la tabla, y la fila 3, el resto.
- Añadimos a la segunda columna de la última fila un **FlowPane** y en su pestaña **Properties** buscamos **Align** y seleccionamos **CENTER_RIGHT**. En la pestaña **Layout** le pondremos un **HGap** de 10
- Añadimos dos **Button** al **FlowPane** y les pondremos de rótulos “Borrar” y “Guardar”. Como id, les pondremos **btnBorrar** y **btnGuardar**.
- Por último, en la sección **Controller** buscamos **Controller Class** y escribimos **ControladorFrmGrados**

Vamos a dar ahora los pasos para diseñar **frmAsignaturas**:

- Añadimos un **GridPane** y al igual que antes, le ponemos 20 de padding en todas las direcciones
- Añadimos una fila más al **GridPane**, de forma que en total tenga 4 filas.
- Añadimos a las 3 primeras filas, en sus primeras columnas, **Label** con estos letreros: “Nombre”, “Curso” y “Créditos”.
- Añadimos a las 3 primeras filas, en sus segundas columnas, **TextField** con estos id: **txtNombre**, **txtCurso** y **txtCreditos**
- En la segunda columna de la última fila añadimos un **Button** que ponga “Guardar” y su id sea **btnGuardar**. En **Layout** buscamos **HAlignment** y le pondremos **RIGHT**
- Modificamos el tamaño de la ventana y las guías de las columnas del **GridPane** para que el aspecto de la ventana sea el correcto y todo quede correctamente ajustado a su tamaño.
- En **Controller class** ponemos **ControladorFrmAsignaturas**.

Podemos crear en este momento un paquete llamado **dam.grados.infraestructura.ui** y añadirle dos clases vacías llamadas **ControladorFrmGrados** y **ControladorFrmAsignaturas** para los controladores. Hecho esto, pasaremos a crear la clase de la aplicación.

7.11.- Creación de la clase de la aplicación

Añadimos al proyecto un paquete llamado **dam.grados.infraestructura.application** y en él una aplicación JavaFX llamada **ProgramaGrados**. Haremos que en su método **start** se cargue la vista **frmGrados**. También aprovecharemos dicho lugar para inicializar el objeto **Context** que usará el programa, usando como origen de datos **BASEDATOS_JDBC**. A partir de este momento, cualquier clase o ventana que necesite el objeto **Context**, solo tendrá que llamar al método estático **getInstancia** de la clase **Context**. Dicho método nos garantiza que el objeto **Context** obtenido es siempre el mismo y no se crean objetos **Context** duplicados.

Aprovecharemos también para sobrescribir el método **stop** de la aplicación. Ese método es llamado cuando la aplicación JavaFX va a terminar, y lo usaremos para cerrar el objeto **Context**.

```

1. public class ProgramaGrados extends Application {
2.     public static void main(String[] args) {
3.         launch(args);
4.     }
5.     @Override
6.     public void start(Stage primaryStage) {
7.         try {
8.             Context.inicializar(TipoOrigenDatos.BASEDATOS_JDBC);
9.             FXMLLoader fxmlLoader = new FXMLLoader(ClassLoader.getResource("frmGrados.fxml"));
10.            primaryStage.setTitle("Carreras universitarias");
11.            primaryStage.setScene(new Scene(fxmlLoader.load()));
12.            primaryStage.show();
13.        } catch (IOException ex) {
14.            System.out.println("Error al inicializar la aplicación: "+ex.getMessage());
15.        }
16.    }
17.    @Override
18.    public void stop(){ Context.getInstance().cerrar(); }
19. }

```

7.12.- Programación de los controladores

En primer lugar usaremos SceneBuilder para asociar métodos a los eventos que se realicen sobre los objetos de la interfaz. Para ello, abriremos la vista **frmGrados** y seguiremos estos pasos:

- Seleccionamos **btnGuardar** y desplegamos la pestaña **Code**. Buscamos **onAction** y escribimos **btnGuardarPulsado**, que es el nombre del método que se insertará en el controlador, y que será llamado cuando se pulse el botón.
- Hacemos lo mismo con **btnBorrar**, llamando al método **btnBorrarPulsado**
- Seleccionamos **lstGradosUniversitarios** y en **Code** buscamos **on Mouse Clicked** y escribimos **lstGradosUniversitarios_GradoSeleccionado**. De esta forma, hacemos que al pulsar el ratón sobre la lista de grados, se active el método **lstGradosUniversitarios_GradoSeleccionado**⁹.

En **frmAsignaturas** lo único que hay que hacer es asociar un método al botón de guardar, de la misma forma que ya hemos visto para **frmGrados**. Le llamaremos **btnGuardarPulsado**.

Para programar los controladores, crearemos las clases **ControladorFrmGrados** y **ControladorFrmAsignaturas**, y pegaremos en ellas el contenido inicial que nos da SceneBuilder. De esa forma, el código inicial de **ControladorFrmAsignaturas** que nos da SceneBuilder es:

```

1. public class ControladorFrmAsignaturas {
2.     @FXML
3.     private Button btnGuardar;
4.     @FXML
5.     private TextField txtCreditos;
6.     @FXML
7.     private TextField txtCurso;
8.     @FXML
9.     private TextField txtNombre;
10.    @FXML
11.    void btnGuardarPulsado(ActionEvent event) {
12.    }
13. }

```

⁹ Esta forma de detectar cuándo cambia el elemento seleccionado en la lista no es óptima, ya que solo funciona cuando se pulsa el ratón en la lista. Para detectar con precisión el cambio, hay que usar **observables**

Vamos a hacer cambios en **ControladorFrmGrados**. Vemos que hay algunos ? que SceneBuilder ha puesto porque no sabe qué clases de nuestro modelo se mostrarán en las listas y la tabla. Tenemos, manualmente, que sustituir los ? por los tipos adecuados.

- El ListView muestra objetos de tipo Grado y por tanto, cambiamos ? por ListView<Grado>
- El TableView muestra objetos de tipo Asignatura, y por tanto, será TableView<Asignatura>
- En la tabla, la columna del nombre muestra un String con el nombre de la asignatura. Por tanto, le pondremos TableColumn<Asignatura,String>.
- De igual forma, puesto que el número de curso y los créditos son números enteros, le pondremos a las columnas de curso y créditos TableColumn<Asignatura,Integer>

```
1. public class ControladorFrmGrados {
2.     @FXML
3.     private Button btnBorrar;
4.     @FXML
5.     private Button btnGuardar;
6.     @FXML
7.     private ListView<Grado> lstGradosUniversitarios;
8.     @FXML
9.     private TextField txtCreditos;
10.    @FXML
11.    private TextField txtDuracion;
12.    @FXML
13.    private TextField txtNombre;
14.    @FXML
15.    private TableView<Asignatura> tabAsignaturas;
16.    @FXML
17.    private TableColumn<Asignatura,String> colNombre;
18.    @FXML
19.    private TableColumn<Asignatura,Integer> colCreditos;
20.    @FXML
21.    private TableColumn<Asignatura,Integer> colDuracion;
22.    @FXML
23.    void btnBorrarPulsado(ActionEvent event) { }
24.    @FXML
25.    void btnGuardarPulsado(ActionEvent event) { }
26.    @FXML
27.    void lstGradosUniversitarios_GradoSeleccionado(MouseEvent event) { }
28. }
```

Por último, vamos a añadir una variable de instancia llamada **gradosService**, que almacenará el objeto **GradosService** que tenemos configurado en el **Context**. En realidad, esta variable no es necesaria, porque siempre podemos acceder al objeto GradosService de esta forma:

```
GradosService gradosService = Context.getInstance().getGradosService();
```

Sin embargo, como usaremos el GradosService en todos los métodos y escribir lo anterior sería muy pesado, por facilitarnos la vida haremos una variable de instancia que lo almacene. También vamos a crear una variable de instancia para guardar el objeto Grado que el usuario haya seleccionado cuando pulsa en la lista de la izquierda.

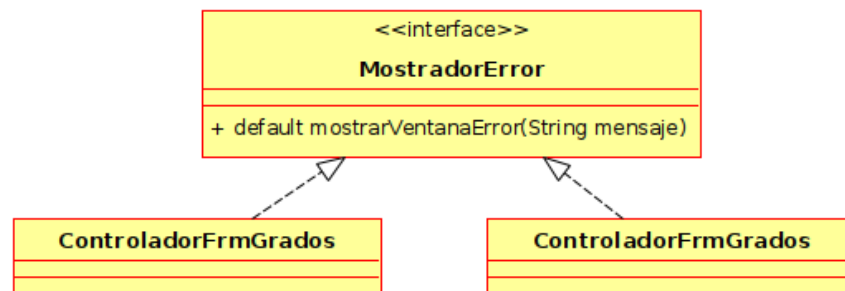
```
1. public class ControladorFrmGrados implements Initializable,MostradorError {
2.     private GradosService gradosService = Context.getInstance().getGradosService();
3.     private Grado gradosSeleccionado;
4.     // resto de la clase omitido
5. }
```

7.12.1.- Gestión común de los mensajes de error

Cuando hacemos una aplicación, debemos preguntarnos cómo se gestionarán los mensajes de error que reciba el usuario. Hay muchas alternativas, como mostrar ventanas emergentes, navegar a una pantalla específica de error, volvar los errores en un archivo (log), etc.

En este proyecto vamos a optar por la solución de que cada error que se produzca, se muestre en una ventana de error. Por tanto, pondremos a los controladores un método llamado **mostrarVentanaError** que abrirá una ventana emergente con un mensaje de error.

Puesto que ese método será el mismo en ambos controladores, para evitar copiar y pegar código (ya sabemos lo malo que es tener código duplicado), haremos una interfaz llamada **MostradorError** con un método default que tenga dicho código, de manera que los dos controladores implementen dicha interfaz, y así lo reciban¹⁰.



Para mostrar una ventana emergente se usa la clase **Alert**, cuyo constructor recibe una constante con el tipo de mensaje (solo influye en el icono que se muestra) y el mensaje de error. El método **show** de dicho objeto, muestra la ventana emergente.

```
1. public interface MostradorError {
2.     public default void mostrarVentanaError(String mensaje){
3.         new Alert(Alert.AlertType.ERROR,mensaje).show();
4.     }
5. }
```

El único paso que falta será hacer que los controladores la implementen. Por ejemplo:

```
1. public class ControladorFrmAsignaturas implements MostradorError{
2.     // resto omitido
3. }
```

7.12.2.- Creación de un nuevo grado

A continuación, vamos a hacer que se puedan guardar grados cuyo nombre y duración escribimos en sus cuadros de texto.

¹⁰ Cuando una interfaz define métodos default que luego se “mezclan” con los de las clases que lo implementan, aumentando la funcionalidad de las clases, la interfaz es una **mixing interface**

El método del controlador que se activa cuando se pulsa el botón de guardar es **btnGuardarPulsado**, así que allí recogeremos el valor de los cuadros de texto, que será un String. Como en el campo de duración solo puede haber un número positivo, usaremos el método **matches** del String introducido para ver si cumple la expresión regular que definen los números positivos. Si falla, se abrirá una ventana emergente con un mensaje de error.

Una vez validado todo, usaremos **gradosService** para crear un nuevo grado en la base de datos. En caso de guardarse correctamente, añadiremos el objeto **Grado** obtenido a la lista **lstGradosUniversitarios**. Para ello, tendremos en cuenta que **lstGradosUniversitarios** guarda en su interior un **List<Grado>** al que accederemos con **getItems**, y con **add** le añadimos el objeto **Grado**.

Finalmente, llamamos a un método privado auxiliar **reiniciarFormulario** que añadiremos con el fin de que tras grabar un nuevo grado, la pantalla quede igual que al inicio del programa. Esto implicará borrar el contenido de los cuadros de texto, la tabla de asignaturas y poner a null el grado que haya sido seleccionado en la lista (si lo hay). Para borrar la tabla de asignaturas, bastará con obtener la lista de sus filas con su método **getItems** y llamar al método **clear** para borrarlas.

```
1. private void btnGuardarPulsado(){
2.     if(txtDuracion.getText().matches("\\d+")) {
3.         String nombre = txtNombre.getText();
4.         int duracion = Integer.parseInt(txtDuracion.getText());
5.         Grado g=gradosService.nuevoGrado(nombre,duracion);
6.         if(g!=null) {
7.             lstGradosUniversitarios.getItems().add(g);
8.             this.reiniciarFormulario();
9.         }else{
10.            mostrarVentanaError("Error al crear el grado");
11.        }
12.    }else {
13.        mostrarVentanaError("La duración debe ser un número válido");
14.    }
15. }
16. private void reiniciarFormulario(){
17.     this.txtNombre.setText("");
18.     this.txtDuracion.setText("");
19.     this.txtCreditos.setText("");
20.     this.btnGuardar.setText("Guardar");
21.     this.tabAsignaturas.getItems().clear();
22.     this.gradoSeleccionado = null;
23. }
```

7.12.3.- Mostrar el detalle del grado seleccionado en el ListView

A continuación, programamos el método **lstGradosUniversitarios_GradoSeleccionado** de forma que se recupere del ListView el objeto **Grado** seleccionado en él, y se muestren sus datos en los cuadros de texto. Para obtener el objeto seleccionado en el ListView llamaremos primero a su método **getSelectionModel**, que nos devuelve un objeto que contiene toda la información posible sobre lo que esté seleccionado en el ListView, y a su método **getSelectedItem**.

Una vez que tenemos el objeto **Grado** seleccionado, lo guardaremos en la variable de instancia **gradoSeleccionado** para que así lo tengamos disponible a la hora de hacer cosas con él, como por ejemplo modificarlo o borrarlo. También cambiaremos el rótulo de **btnGuardar**, para que ponga "Actualizar", ya que en ese momento, hay un grado seleccionado y esa sería su función.


```

1. private void lstGradosUniversitarios_GradoSeleccionado(MouseEvent event) {
2.     this.gradoSeleccionado = lstGradosUniversitarios.getSelectionModel().getSelectedItem();
3.     if(this.gradoSeleccionado!=null){
4.         txtNombre.setText(gradoSeleccionado.getNombre());
5.         txtDuracion.setText(""+gradoSeleccionado.getDuracion());
6.         txtCreditos.setText(""+gradoSeleccionado.getCreditos());
7.         List<Asignatura> asignaturas = gradoSeleccionado.getAsignaturas();
8.         tabAsignaturas.getItems().setAll(asignaturas);
9.         btnGuardar.setText("Actualizar");
10.    }
11. }

```

7.12.4.- Borrar un grado

Ahora vamos a programar el método **btnBorrarPulsado** para que se borre el objeto que esté seleccionado en el ListView. Para ello, intentaremos primero obtener dicho objeto de la misma forma que en el apartado anterior, y en caso de que sea posible, se lo pasaremos a **gradosService** para que lo borre. En caso de que se pueda borrar, lo eliminamos también del ListView y reestablecemos la ventana a su estado inicial, haciendo, entre otras cosas, que el ListView no tenga nada seleccionado.

Suele ser habitual a la hora de borrar pedir una confirmación al usuario. Lo conseguiremos creando un objeto **Alert** de tipo “confirmación”. Tras configurar su cabecera (**setHeaderText**) y su mensaje (**setContentText**) lo mostraremos con **showAndWait**. Este método devuelve un **Optional** con el botón que ha sido pulsado, lo que servirá para actuar en caso de que se haya pulsado “ok”.

```

1. private void btnBorrarPulsado(ActionEvent event) {
2.     if(this.gradoSeleccionado!=null){
3.         Alert a = new Alert(Alert.AlertType.CONFIRMATION);
4.         a.setHeaderText("Confirmación de borrado");
5.         a.setContentText("¿Desea borrar el grado "+this.gradoSeleccionado.getNombre()+"?");
6.         a.showAndWait().ifPresent( boton -> {
7.             if(boton == ButtonType.OK){
8.                 gradosService.borrar(this.gradoSeleccionado);
9.                 lstGradosUniversitarios.getItems().remove(this.gradoSeleccionado);
10.                this.reiniciarFormulario();
11.            }
12.        });
13.    }
14. }
15. }

```

7.12.5.- Actualizar un grado

Si ejecutamos el programa en este momento podremos ver que podemos crear nuevos grados que se añaden al ListView, también que al seleccionar un grado del ListView aparecen sus características en la parte derecha de la pantalla, y que podemos borrar grados. Sin embargo, lo ideal sería que si seleccionamos un grado y modificamos sus características, al pulsar el botón de guardar, este se actualizara. Ahora mismo, lo que se haría sería crear un nuevo grado. Por tanto, vamos a modificar el método **btnGuardarPulsado** para que, en caso de haber un objeto seleccionado en el ListView, este sea modificado, en lugar de que se cree un grado nuevo.

Para que el código de **btnGuardarPulsado** no quede muy grande y mezcle cosas diferentes (crear un grado y modificar un grado), crearemos dos métodos auxiliares que llamaremos **crearNuevoGrado** y **actualizarGrado** de forma que se llame a uno u otro, según **gradoSeleccionado** sea null. Dicha comprobación se hará en **btnGuardarPulsado**

```

1. private void btnGuardarPulsado(ActionEvent event){
2.     if(this.gradoSeleccionado==null){
3.         this.nuevoGrado();
4.     }else{
5.         this.actualizarGrado();
6.     }
7. }
8. private void nuevoGrado(){
9.     if(txtDuracion.getText().matches("\\d+")) {
10.         String nombre = txtNombre.getText();
11.         int duracion = Integer.parseInt(txtDuracion.getText());
12.         Grado g=gradosService.nuevoGrado(nombre,duracion);
13.         if(g!=null) {
14.             lstGradosUniversitarios.getItems().add(g);
15.             this.reiniciarFormulario();
16.         }else{
17.             mostrarVentanaError("Error al crear el grado");
18.         }
19.     }else {
20.         mostrarVentanaError("La duración debe ser un número válido");
21.     }
22. }
23. private void actualizarGrado(){
24.     // aquí actualizaremos el grado seleccionado con los datos de la pantalla
25. }

```

Para programar el método que actualiza el grado seleccionado, recogemos los datos de la pantalla, validamos que la duración sea un número positivo, y modificamos con los setters el objeto **gradoSeleccionado**. Para que la actualización del objeto se haga efectiva en la base de datos, usamos el método **actualizar** de **gradosService**

```

1. private void actualizarGrado(){
2.     if(txtDuracion.getText().matches("\\d+")) {
3.         this.gradoSeleccionado.setNombre(txtNombre.getText());
4.         this.gradoSeleccionado.setDuracion(Integer.parseInt(txtDuracion.getText()));
5.         gradosService.actualizar(this.gradoSeleccionado);
6.     }else{
7.         mostrarVentanaError("La duración debe ser un número válido");
8.     }
9. }

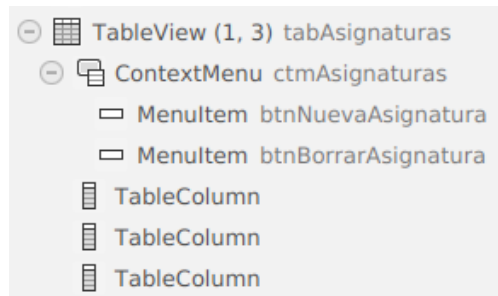
```

7.12.6.- Añadir un menú contextual a la tabla

Vamos ahora a añadir un menú contextual a la **tabAsignaturas**, que tendrá dos ítems: “Nueva asignatura” y “Borrar asignatura”. El menú contextual es un objeto de la clase **ContextMenu**, que contiene objetos de tipo **MenuItem** con las opciones. Diseñaremos todo con SceneBuilder.

Abrimos **frmGrados** en SceneBuilder y arrastramos dentro de **tabAsignaturas** un **ContextMenu** cuyo id será **ctmAsignaturas**. Dentro de él le ponemos dos **MenuItem** que configuraremos de esta forma:

- Pondremos al primero el id **btNuevaAsignatura**, en **text** le pondremos “Nueva asignatura” y en su evento **on Action** le pondremos **btnNuevaAsignaturaPulsada**
- Pondremos al segundo el id **btBorrarAsignatura**, en **text** le pondremos “Borrar asignatura” y en su evento **on Action** le pondremos **btBorrarAsignaturaPulsada**



Una vez guardado, veremos que el archivo fxml de nuestro controlador da error porque no existen los métodos que hemos definido. Se los añadimos al controlador y no olvidemos anotarlos con **@FXML**

```

1. @FXML
2. public void btnNuevaAsignaturaPulsada(ActionEvent actionEvent) {
3. }
4. @FXML
5. public void btnBorrarAsignaturaPulsada(ActionEvent actionEvent) {
6. }

```

7.12.7.- Abrir la ventana de asignaturas en forma modal

En este paso vamos a hacer que al pulsar el botón contextual para añadir asignaturas, se abra la ventana diseñada en **frmAsignaturas.fxml** en forma modal. Para ello nos vamos al método **btnNuevaAsignaturaPulsada**, que es donde programaremos lo que pasa cuando se pulsa la opción para añadir una nueva asignatura, y haremos los siguientes pasos:

- Usamos la clase **FXMLLoader** para cargar la vista **frmAsignaturas.fxml**
- Creamos un objeto **Scene** a partir del objeto que nos devuelve el método **load** de **FXMLLoader**, como ya hemos hecho antes.
- Creamos un objeto **Stage**, que será la ventana modal, y lo configuramos de esta forma:
 - Le ponemos la escena que acabamos de crear
 - Le ponemos de título a la ventana el mensaje “Nueva asignatura”
 - Usaremos su método **initModality** para pasarle la constante **Modality.WINDOW_MODAL**, que indica que esa ventana se mostrará en forma modal (no se puede volver a la ventana padre hasta cerrarla)
 - Por último, usaremos el método **initOwner** para pasarle a la ventana su ventana padre, que es la ventana principal de la aplicación. Podemos obtener dicha ventana cogiendo cualquier elemento de la interfaz (por ejemplo, el botón de borrar) y llamar a su método a **getScene()** para obtener su escena y después a **getWindow()** para obtener su ventana.
- Hecho todo esto, llamaremos al método **showAndWait()** de la Scene del apartado anterior para que esta se abra en forma modal con respecto a la ventana principal de la aplicación.

```

1. private void btnNuevaAsignaturaPulsada(ActionEvent event) {
2.     if(this.gradoSeleccionado!=null){
3.         try {
4.             FXMLLoader fxmLoader = new FXMLLoader(ClassLoader.getResource("frmAsignaturas.fxml"));
5.             Scene e = new Scene(fxmLoader.load());
6.             ControladorFrmAsignaturas controlador = fxmLoader.getController();
7.             Stage s = new Stage();
8.             s.setTitle("Nueva asignatura");
9.             s.setScene(e);
10.            s.initModality(Modality.WINDOW_MODAL);
11.            s.initOwner(btnBorrar.getScene().getWindow());
12.            s.showAndWait();
13.            Asignatura a = controlador.getAsignaturaCreada();
14.            tabAsignaturas.getItems().add(a);
15.        }catch(IOException e){
16.            mostrarVentanaError("No se pudo abrir el formulario para crear asignaturas");
17.        }
18.    }
19. }

```

7.12.8.- Pasar un objeto de la ventana principal a la ventana modal

La ventana modal va a crear un objeto **Asignatura** y lo tiene que añadir al grado seleccionado en la ventana principal. Esto nos indica que tenemos que pasar el objeto **Grado** seleccionado de la ventana principal a la ventana modal. Para conseguirlo:

- Creamos una variable de instancia **gradoSeleccionado** en **ControladorFrmAsignaturas**
- Ponemos un **setter** a esa variable de instancia

```

1. public class ControladorFrmAsignaturas implements MostradorError{
2.     private Grado gradoSeleccionado;
3.     public void setGradoSeleccionado(Grado g){
4.         this.gradoSeleccionado=g;
5.     }
6.     // resto omitido
7. }

```

- En el método **btnNuevaAsignaturaPulsada** de **ControladorFrmGrados**, cuando vamos a abrir la ventana modal, justo en el punto donde obtenemos el controlador, llamamos al setter que acabamos de crear y le pasamos la variable de instancia **gradoSeleccionado**

```

1. private void btnNuevaAsignaturaPulsada(ActionEvent event) {
2.     if(this.gradoSeleccionado!=null){
3.         try {
4.             FXMLLoader fxmLoader = new FXMLLoader(ClassLoader.getResource("frmAsignaturas.fxml"));
5.             Scene e = new Scene(fxmLoader.load());
6.             ControladorFrmAsignaturas controlador = fxmLoader.getController();
7.             controlador.setGradoSeleccionado(this.gradoSeleccionado);
8.             Stage s = new Stage();
9.             s.setTitle("Nueva asignatura");
10.            s.setScene(e);
11.            s.initModality(Modality.WINDOW_MODAL);
12.            s.initOwner(btnBorrar.getScene().getWindow());
13.            s.showAndWait();
14.            Asignatura a = controlador.getAsignaturaCreada();
15.            tabAsignaturas.getItems().add(a);
16.        }catch(IOException e){
17.            mostrarVentanaError("No se pudo abrir el formulario para crear asignaturas");
18.        }
19.    }
20. }

```

De esta forma, el grado seleccionado pasa a la ventana modal.

7.12.9.- Pasar un objeto de la ventana modal a la ventana principal

Cuando usamos la ventana modal y pulsamos su botón **btnGuardarPulsado**, se graba en la base de datos un objeto **Asignatura** con los datos que hay en dicha ventana. Este objeto deberá ser pasado a la ventana principal para que esta lo muestre en la tabla **tabAsignaturas**. Para hacer eso, la idea es la siguiente:

- En **ControladorFrmAsignaturas** creamos una variable de instancia llamada **asignaturaCreada**
- Le ponemos un **getter** a esa variable de instancia

```
1. public class ControladorFrmAsignaturas implements MostradorError{
2.     private Asignatura asignaturaCreada;
3.     // resto omitido
4. }
```

- En **ControladorFrmAsignaturas** programamos el método **btnGuardarPulsado** de forma que se recojan los datos de la pantalla, se validen, y con **gradosService** se cree la asignatura buscada. Si todo va bien, hacemos que se cierre la ventana modal y si no, se muestre una ventana emergente de error. Es muy importante guardar en la variable de instancia **asignaturaCreada** el objeto **Asignatura** que devuelve el método que crea la asignatura.

```
1. private void btnGuardarPulsado(ActionEvent event) {
2.     GradosService gradosService = Context.getInstancia().getGradosService();
3.     if(this.txtCreditos.getText().matches("\\d+")){
4.         if(this.txtCurso.getText().matches("\\d+")){
5.             int nc = Integer.parseInt(this.txtCurso.getText());
6.             int cr = Integer.parseInt(this.txtCreditos.getText());
7.             this.asignaturaCreada =
8.                 gradosService.nuevaAsignatura(this.gradoSeleccionado,txtNombre.getText(),nc,cr);
9.             if(this.asignaturaCreada!=null){
10.                ((Stage)this.btnGuardar.getScene().getWindow()).close(); // así se cierra la ventana modal
11.            }else{
12.                this.mostrarVentanaError("Error al crear la asignatura");
13.            }
14.        }else{
15.            this.mostrarVentanaError("El curso debe ser un número válido");
16.        }
17.    }else{
18.        this.mostrarVentanaError("Los créditos deben ser un número válido");
19.    }
20. }
```

- En **ControladorFrmGrados**, una vez que se cierra la ventana modal (justo tras la llamada a **showAndWait**) llamamos al getter que nos devuelve la asignatura creada y la añadimos a la lista de objetos de la tabla **tabAsignaturas**

```
1. FXMLLoader fxmLoader = new FXMLLoader(ClassLoader.getResource("frmAsignaturas.fxml"));
2. Scene e = new Scene(fxmLoader.load());
3. ControladorFrmAsignaturas controlador = fxmLoader.getController();
4. controlador.setGradoSeleccionado(this.gradoSeleccionado);
5. Stage s = new Stage();
6. s.setTitle("Nueva asignatura");
7. s.setScene(e);
8. s.initModality(Modality.WINDOW_MODAL);
9. s.initOwner(btnBorrar.getScene().getWindow());
10. s.showAndWait();
11. Asignatura a = controlador.getAsignaturaCreada();
12. tabAsignaturas.getItems().add(a);
```

7.12.10.- Configurar las columnas de la tabla

En el punto anterior hemos añadido la asignatura creada en la ventana modal a **tabAsignaturas**. Sin embargo, al ejecutar el programa vemos que al añadir asignaturas parece que se añaden filas a la tabla, pero no se muestra nada en ellas. Eso se debe a que las columnas de la tabla aún no han sido configuradas y por tanto, cada columna no sabe qué información sobre la asignatura tiene que mostrar. O sea: cada fila de la tabla contiene un objeto **Asignatura**, y las columnas de la tabla deben ser configuradas para indicar qué deberán mostrar.

Esto se hace llamando al método **setCellValueFactory**, al que se pasa un objeto **PropertyValueFactory**, que en su constructor recibe un String con el nombre de la variable de instancia de la clase **Asignatura** que se desea mostrar en la columna¹¹.

En **ControladorFrmGrados** crearemos un método llamado **configurarColumnasTabla** donde configuraremos las columnas de la tabla, tal y como hemos descrito:

```
1. private void configurarColumnasTabla(){
2.     colNombre.setCellValueFactory(new PropertyValueFactory<>("nombre"));
3.     colCurso.setCellValueFactory(new PropertyValueFactory<>("numeroCurso"));
4.     colCreditos.setCellValueFactory(new PropertyValueFactory<>("creditos"));
5. }
```

7.12.11.- Configurar el inicio de la aplicación

Vamos ahora a hacer que al iniciarse la aplicación ocurran una serie de cosas que son importantes:

- Se llame al método **configurarColumnasTabla**, para que así las columnas muestren los datos apropiados cuando se añade un objeto **Asignatura** a la tabla.
- Se carguen los grados que hay en la base de datos y se muestren en la lista

Es posible añadir a un controlador un método que se ejecutará solo una vez, en el momento en que el controlador es cargado, y ese es el lugar donde programar las acciones anteriores.

Hacemos que **ControladorFrmGrados** implemente la interfaz **Initializable** y eso nos llevará a sobrescribir el método **intialize**. En él es donde haremos las tareas anteriores. Para que dicho código sea fácil de leer, la parte de cargar los datos y mostrarla en la lista se hará en un método separado llamado **cargarDatosIniciales**.

```
1. public void initialize(URL url, ResourceBundle resourceBundle) {
2.     this.configurarColumnasTabla();
3.     this.cargarDatosIniciales();
4. }
```

¹¹ Cuando se desea mostrar en la columna un valor que no se corresponde con una variable de instancia, es necesario pasar a ese método un **SimpleObjectProperty** que recibe una expresión lambda que transforma un objeto **Asignatura** en el valor que se desea mostrar en la columna.

El método que carga los datos de la base de datos y los muestra en la lista es:

```
1. private void cargarDatosIniciales(){
2.     List<Grado> grados=Context.getInstance().getGradosService().consultarGrados();
3.     lstGradosUniversitarios.getItems().setAll(grados);
4. }
```

7.12.12.- Borrar la asignatura de la tabla

Por último, para borrar la asignatura que esté seleccionada en la tabla, programaremos el método **btnBorrarAsignaturaPulsada** de forma que se recupere el objeto seleccionado y se borre.

```
1. private void btnBorrarAsignaturaPulsada(ActionEvent event) {
2.     Asignatura a = tabAsignaturas.getSelectionModel().getSelectedItem();
3.     if(a!=null){
4.         if(gradosService.borrar(a)){
5.             tabAsignaturas.getItems().remove(a);
6.         }else{
7.             mostrarVentanaError("Error al borrar la asignatura "+a.getNombre());
8.         }
9.     }
10. }
```